

```
"""Domain contracts for the modular weekly weekly image prescan."""
from .domain import (
    CollectionContext,
    CollectionResult,
    CurationResult,
    DiscoveryMethod,
    EventType,
    FailureRecord,
    FailureStage,
    HistoricalImage,
    ImageCandidate,
    ImageNeed,
    Issue,
    IssueDraft,
    NewsDraft,
    NewsItem,
    NewsResult,
    OutputManifest,
    PipelineResult,
    ReviewReason,
    ScoreBreakdown,
    SearchResult,
    SourceRef,
    SourceType,
    candidate_id_for,
    news_id_for,
)
__all__ = [
    "CollectionContext",
    "CollectionResult",
    "CurationResult",
    "DiscoveryMethod",
    "EventType",
    "FailureRecord",
    "FailureStage",
    "HistoricalImage",
    "ImageCandidate",
    "ImageNeed",
    "Issue",
    "IssueDraft",
    "NewsDraft",
    "NewsItem",
    "NewsResult",
    "OutputManifest",
    "PipelineResult",
    "ReviewReason",
    "ScoreBreakdown",
    "SearchResult",
    "SourceRef",
    "SourceType",
```

```

        "candidate_id_for",
        "news_id_for",
    ]
    from .cli import main
    raise SystemExit(main())
    """Stable, validated data contracts shared by all prescan modules."""
    from __future__ import annotations
    import hashlib
    import re
    import unicodedata
    from datetime import datetime, timezone
    from enum import Enum
    from pathlib import Path
    from typing import Any, Literal, Protocol
    from urllib.parse import urlsplit, urlunsplit
    from pydantic import BaseModel, ConfigDict, Field, field_validator, model_validator
    class _StringEnum(str, Enum):
        @classmethod
        def _missing_(cls, value: object):
            return cls.UNKNOWN
    class EventType(_StringEnum):
        NEW_TITLE = "new_title"
        RELEASE = "release"
        DEMO = "demo"
        UPDATE = "update"
        EVENT = "event"
        GOODS = "goods"
        LOCALIZATION = "localization"
        UNKNOWN = "unknown"
    class VideoStatus(_StringEnum):
        DISCOVERED = "discovered"
        DOWNLOADING = "downloading"
        DOWNLOADED = "downloaded"
        SKIPPED = "skipped"
        FAILED = "failed"
        UNKNOWN = "unknown"
    class SourceType(_StringEnum):
        OFFICIAL_SITE = "official_site"
        OFFICIAL_X = "official_x"
        STEAM = "steam"
        DIRECT_IMAGE = "direct_image"
        VIDEO = "video"
        UNVERIFIED = "unverified"
        UNKNOWN = "unknown"
    class ImageType(_StringEnum):
        GAME_CG = "game_cg"
        GAMEPLAY_SCREENSHOT = "gameplay_screenshot"
        KEY_VISUAL = "key_visual"
        CHARACTER_ART = "character_art"
        COVER = "cover"

```

```

ANNOUNCEMENT_ART = "announcement_art"
GOODS = "goods"
LOGO = "logo"
BANNER = "banner"
UI = "ui"
PHOTO = "photo"
UNKNOWN = "unknown"

class ImageCurationStatus(_StringEnum):
    SELECTED = "selected"
    UNSELECTED = "unselected"
    INVALID = "invalid"
    UNKNOWN = "unknown"

class ImageNeed(_StringEnum):
    EXPLICIT_NEW_IMAGE = "explicit_new_image"
    EVENT_IMAGE = "event_image"
    GENERIC_EDITORIAL = "generic_editorial"
    UNKNOWN = "unknown"

class DiscoveryMethod(_StringEnum):
    DOCUMENT = "document"
    HISTORY = "history"
    SAME_DOMAIN = "same_domain"
    BRAVE = "brave"
    DDGS = "ddgs"
    MANUAL = "manual"
    UNKNOWN = "unknown"

class ReviewReason(_StringEnum):
    X_SOURCE = "x_source"
    AGE_GATE = "age_gate"
    DYNAMIC_PAGE = "dynamic_page"
    UNKNOWN_PUBLISH_TIME = "unknown_publish_time"
    UNCERTAIN_MATCH = "uncertain_match"
    CLOSE_SCORES = "close_scores"
    NETWORK_RESTRICTED = "network_restricted"
    FALLBACK_OLD_MATERIAL = "fallback_old_material"
    HISTORICAL_DUPLICATE = "historical_duplicate"
    ADULT_OR_UNKNOWN = "adult_or_unknown"
    IMAGE_TYPE_REVIEW = "image_type_review"
    IMAGE_QUALITY_REVIEW = "image_quality_review"

class FailureStage(_StringEnum):
    PARSE = "parse"
    ANALYZE = "analyze"
    RESOLVE = "resolve"
    COLLECT = "collect"
    DOWNLOAD = "download"
    CURATE = "curate"
    OUTPUT = "output"
    HISTORY = "history"
    UNKNOWN = "unknown"

def _normalized_text(value: str) -> str:
    value = unicodedata.normalize("NFKC", value)

```

```

    return re.sub(r"\s+", " ", value).strip().casefold()
def _normalized_url(value: str) -> str:
    parts = urlsplit(value.strip())
    return urlunsplit((parts.scheme.casefold(), parts.netloc.casefold(), parts.path, parts.query, ""))
def news_id_for(issue_id: str, sequence: int, title: str) -> str:
    raw = f"{issue_id}\0{sequence}\0{_normalized_text(title)}".encode("utf-8")
    return hashlib.sha256(raw).hexdigest()[:16]
def candidate_id_for(image_url: str, news_id: str | None = None) -> str:
    """Return a stable image identity scoped to its news item when available."""
    key = f"{news_id}\0{_normalized_url(image_url)}" if news_id else _normalized_url(image_url)
    return hashlib.sha256(key.encode("utf-8")).hexdigest()[:16]
def video_candidate_id_for(video_url: str) -> str:
    """Return a stable id for a video URL after removing fragments."""
    return hashlib.sha256(_normalized_url(video_url).encode("utf-8")).hexdigest()[:16]
def video_id_for(video_url: str) -> str:
    """Backward-friendly alias for callers that use the shorter name."""
    return video_candidate_id_for(video_url)
def _aware(value: datetime | None) -> datetime | None:
    if value is not None and value.tzinfo is None:
        raise ValueError("datetime must include timezone")
    return value
class ContractModel(BaseModel):
    model_config = ConfigDict(extra="forbid", validate_assignment=True)
class NewsDraft(ContractModel):
    sequence: int = Field(ge=1)
    section: str = Field(min_length=1)
    title: str = Field(min_length=1)
    author: str | None = None
    body: str
    source_urls: list[str] = Field(default_factory=list)
class IssueDraft(ContractModel):
    issue_id: str = Field(min_length=1)
    input_path: str = Field(min_length=1)
    published_at: datetime | None = None
    entries: list[NewsDraft] = Field(default_factory=list)
    _published_at_aware = field_validator("published_at")(_aware)
class NewsItem(ContractModel):
    id: str | None = None
    issue_id: str = Field(min_length=1)
    sequence: int = Field(ge=1)
    section: str = Field(min_length=1)
    title: str = Field(min_length=1)
    author: str | None = None
    body: str
    game_names: list[str] = Field(default_factory=list)
    organizations: list[str] = Field(default_factory=list)
    people: list[str] = Field(default_factory=list)
    event_type: EventType = EventType.UNKNOWN
    event_at: datetime | None = None
    keywords: list[str] = Field(default_factory=list)

```

```

source_urls: list[str] = Field(default_factory=list)
importance: float = Field(default=0.0, ge=0.0, le=1.0)
image_need: ImageNeed = ImageNeed.UNKNOWN
@model_validator(mode="after")
def assign_stable_id(self) -> "NewItem":
    object.__setattr__(self, "id", news_id_for(self.issue_id, self.sequence, self.title))
    return self
_event_at_aware = field_validator("event_at")(_aware)
class Issue(ContractModel):
    schema_version: Literal[1] = Field(default=1, frozen=True)
    issue_id: str = Field(min_length=1)
    input_path: str = Field(min_length=1)
    published_at: datetime | None = None
    news_items: list[NewItem] = Field(default_factory=list)
    _published_at_aware = field_validator("published_at")(_aware)
class SourceRef(ContractModel):
    url: str = Field(min_length=1)
    source_type: SourceType = SourceType.UNKNOWN
    domain: str = Field(min_length=1)
    discovered_via: DiscoveryMethod = DiscoveryMethod.UNKNOWN
    officiality: float = Field(default=0.0, ge=0.0, le=1.0)
    requires_review: bool = False
    review_reasons: list[ReviewReason] = Field(default_factory=list)
    root_url: str | None = None
    parent_url: str | None = None
class ScoreBreakdown(ContractModel):
    relevance: float = Field(ge=0.0, le=1.0)
    freshness: float = Field(ge=0.0, le=1.0)
    source_trust: float = Field(ge=0.0, le=1.0)
    quality: float = Field(ge=0.0, le=1.0)
    duplicate_penalty: float = Field(default=0.0, ge=0.0, le=1.0)
    risk_penalty: float = Field(default=0.0, ge=0.0, le=1.0)
    type_match: float = Field(default=0.0, ge=0.0, le=1.0)
    total: float = Field(ge=0.0, le=100.0)
class ImageCandidate(ContractModel):
    id: str | None = None
    news_id: str = Field(min_length=1)
    image_url: str = Field(min_length=1)
    source_url: str = Field(min_length=1)
    source_type: SourceType = SourceType.UNKNOWN
    image_type: ImageType = ImageType.UNKNOWN
    image_type_confidence: float = Field(default=0.0, ge=0.0, le=1.0)
    published_at: datetime | None = None
    fetched_at: datetime
    width: int | None = Field(default=None, ge=1)
    height: int | None = Field(default=None, ge=1)
    mime_type: str | None = None
    byte_size: int | None = Field(default=None, ge=0)
    sha256: str | None = None
    perceptual_hash: str | None = None

```

```

downloadable: bool = False
local_path: str | None = None
original_path: str | None = None
original_mime_type: str | None = None
original_byte_size: int | None = Field(default=None, ge=0)
original_sha256: str | None = None
original_width: int | None = Field(default=None, ge=1)
original_height: int | None = Field(default=None, ge=1)
output_mime_type: str | None = None
output_byte_size: int | None = Field(default=None, ge=0)
output_sha256: str | None = None
output_width: int | None = Field(default=None, ge=1)
output_height: int | None = Field(default=None, ge=1)
news_source_url: str | None = None
parent_source_url: str | None = None
image_alt: str | None = None
nearby_text: str | None = None
selection_reasons: list[str] = Field(default_factory=list)
animated_source: bool = False
animation_frame_index: int | None = Field(default=None, ge=0)
review_reasons: list[ReviewReason] = Field(default_factory=list)
signals: dict[str, float | int | str | bool] = Field(default_factory=dict)
score: ScoreBreakdown | None = None
selected: bool = False
curation_status: ImageCurationStatus = ImageCurationStatus.UNSELECTED
@model_validator(mode="after")
def assign_stable_id(self) -> "ImageCandidate":
    object.__setattr__(self, "id", candidate_id_for(self.image_url, self.news_id))
    if self.signals.get("invalid_reason") or self.curation_status is ImageCurationStatus.INVALID:
        object.__setattr__(self, "selected", False)
        object.__setattr__(self, "curation_status", ImageCurationStatus.INVALID)
    else:
        object.__setattr__(self, "curation_status", ImageCurationStatus.SELECTED if self.selected else ImageCurationStatus.UNSELECTED)
    return self
    _published_at_aware = field_validator("published_at")(_aware)
    _fetched_at_aware = field_validator("fetched_at")(_aware)
class VideoCandidate(ContractModel):
    id: str | None = None
    news_id: str = Field(min_length=1)
    source_url: str = Field(min_length=1)
    video_url: str = Field(min_length=1)
    title: str = ""
    uploader: str = ""
    duration_seconds: float | None = Field(default=None, ge=0)
    width: int | None = Field(default=None, ge=1)
    height: int | None = Field(default=None, ge=1)
    format: str | None = None
    mime_type: str | None = None
    byte_size: int | None = Field(default=None, ge=0)

```

```

sha256: str | None = None
local_path: str | None = None
downloadable: bool = False
review_reasons: list[str] = Field(default_factory=list)
status: VideoStatus = VideoStatus.DISCOVERED
failure_reason: str | None = None
discovered_via: DiscoveryMethod = DiscoveryMethod.UNKNOWN
@model_validator(mode="after")
def assign_stable_id(self) -> "VideoCandidate":
    object.__setattr__(self, "id", video_candidate_id_for(self.video_url))
    return self

class FailureRecord(ContractModel):
    stage: FailureStage = FailureStage.UNKNOWN
    news_id: str | None = None
    candidate_id: str | None = None
    code: str = Field(min_length=1)
    message: str = Field(min_length=1)
    source_url: str | None = None
    retryable: bool = False
    occurrences: int = Field(default=1, ge=1)
    occurred_at: datetime = Field(default_factory=lambda: datetime.now(timezone.utc))
    _occurred_at_aware = field_validator("occurred_at")(_aware)

class HistoricalImage(ContractModel):
    image_id: str
    sha256: str | None = None
    perceptual_hash: str | None = None
    first_seen_issue: str
    last_seen_issue: str
    local_path: str | None = None

class SearchResult(ContractModel):
    url: str = Field(min_length=1)
    title: str = ""
    snippet: str = ""
    domain: str = ""

class CollectionContext(ContractModel):
    timeout_seconds: float = Field(default=12.0, gt=0)
    max_candidates: int = Field(default=20, ge=1)
    now: datetime = Field(default_factory=lambda: datetime.now(timezone.utc))
    _now_aware = field_validator("now")(_aware)

class CollectionResult(ContractModel):
    candidates: list[ImageCandidate] = Field(default_factory=list)
    video_candidates: list[VideoCandidate] = Field(default_factory=list)
    failures: list[FailureRecord] = Field(default_factory=list)
    manual_review_reasons: list[ReviewReason] = Field(default_factory=list)

class NewsResult(ContractModel):
    news_item: NewsItem
    sources: list[SourceRef] = Field(default_factory=list)
    candidates: list[ImageCandidate] = Field(default_factory=list)
    failures: list[FailureRecord] = Field(default_factory=list)
    recorded_at: datetime = Field(default_factory=lambda: datetime.now(timezone.utc))

```

```

    _recorded_at_aware = field_validator("recorded_at")(_aware)
class CurationResult(ContractModel):
    candidates: list[ImageCandidate] = Field(default_factory=list)
    filtered_candidates: list[ImageCandidate] = Field(default_factory=list)
    failures: list[FailureRecord] = Field(default_factory=list)
    selection_shortfall: int = Field(default=0, ge=0)
class PipelineResult(ContractModel):
    schema_version: Literal[1] = Field(default=1, frozen=True)
    issue: Issue
    candidates: list[ImageCandidate] = Field(default_factory=list)
    videos: list[VideoCandidate] = Field(default_factory=list)
    filtered_candidates: list[ImageCandidate] = Field(default_factory=list)
    failures: list[FailureRecord] = Field(default_factory=list)
    review_required: list[ImageCandidate] = Field(default_factory=list)
    @property
    def all_candidates(self) -> list[ImageCandidate]:
        return [*self.candidates, *self.filtered_candidates]
class OutputManifest(ContractModel):
    schema_version: Literal[1] = Field(default=1, frozen=True)
    issue_id: str = Field(min_length=1)
    output_dir: str = Field(min_length=1)
    image_count: int = Field(default=0, ge=0)
    files: list[str] = Field(default_factory=list)
class HistoryStore(Protocol):
    def known_image(self, sha256: str | None, perceptual_hash: str | None) -> HistoricalImage | None: ...
    .
    def sources_for(self, news_item: NewsItem) -> list[SourceRef]: ...
    def record_news_result(self, result: NewsResult) -> None: ...
"""TOML-backed validation for curation and networking policy."""
from __future__ import annotations
import copy
import sys
import tomllib
from pathlib import Path
from typing import Literal
from pydantic import BaseModel, ConfigDict, Field, model_validator
class _ConfigModel(BaseModel):
    model_config = ConfigDict(extra="forbid")
class ScoringConfig(_ConfigModel):
    relevance: float = Field(ge=0.0, le=1.0)
    freshness: float = Field(ge=0.0, le=1.0)
    source_trust: float = Field(ge=0.0, le=1.0)
    quality: float = Field(ge=0.0, le=1.0)
    duplicate_penalty: float = Field(ge=0.0, le=1.0)
    risk_penalty: float = Field(ge=0.0, le=1.0)
    @model_validator(mode="after")
    def weights_sum_to_one(self) -> "ScoringConfig":
        if abs((self.relevance + self.freshness + self.source_trust + self.quality) - 1.0) > 1e-6:
            raise ValueError("scoring weights must sum to 1")
        return self

```



```

class FilterConfig(_ConfigModel):
    min_width: int = Field(ge=1)
    min_height: int = Field(ge=1)
    min_pixels: int = Field(ge=1)
    max_response_bytes: int = Field(ge=1)
    max_image_bytes: int = Field(ge=1)
class NetworkConfig(_ConfigModel):
    timeout_seconds: float = Field(gt=0)
    max_retries: int = Field(ge=0, le=10)
    user_agent: str = Field(min_length=1)
    page_concurrency: int = Field(default=4, ge=1, le=64)
    page_per_host: int = Field(default=2, ge=1, le=64)
    image_concurrency: int = Field(default=8, ge=1, le=64)
    image_per_host: int = Field(default=8, ge=1, le=64)
    image_processing_threads: int = Field(default=2, ge=1, le=16)
    socialdata_concurrency: int = Field(default=2, ge=1, le=16)
CONCURRENCY_FIELDS = frozenset({
    "page_concurrency", "page_per_host", "image_concurrency", "image_per_host",
    "image_processing_threads", "socialdata_concurrency",
})
class SelectionConfig(_ConfigModel):
    min_images: int = Field(ge=0)
    max_images: int = Field(ge=1)
    per_news_max: int = Field(ge=1)
    minimum_score: float = Field(ge=0.0, le=100.0)
    type_limits: dict[str, int] = Field(default_factory=lambda: copy.deepcopy(DEFAULT_TYPE_LIMITS))
    @model_validator(mode="after")
    def type_limits_are_valid(self) -> "SelectionConfig":
        if any(not isinstance(value, int) or value < 0 for value in self.type_limits.values()):
            raise ValueError("selection.type_limits values must be non-negative integers")
        return self
    @model_validator(mode="after")
    def limits_are_ordered(self) -> "SelectionConfig":
        if self.min_images > self.max_images:
            raise ValueError("min_images cannot exceed max_images")
        return self
class VideoConfig(_ConfigModel):
    enabled: bool = True
    max_height: int = Field(default=1080, ge=1)
    max_duration_seconds: int = Field(default=600, gt=0)
    max_file_bytes: int = Field(default=1073741824, gt=0)
    max_per_news: int = Field(default=3, ge=0)
    preferred_container: Literal["mp4", "webm"] = "mp4"
    # Optional directory or ffmpeg executable. FFMPEG_LOCATION takes
    # precedence at runtime; an empty value means automatic discovery.
    ffmpeg_location: str | None = None
class SearchConfig(_ConfigModel):
    max_results: int = Field(ge=1)
    same_domain_depth: int = Field(ge=0, le=3)
    max_candidates_per_source: int = Field(ge=20, le=2000)

```

```

class VisualAnalysisConfig(_ConfigModel):
    enabled: bool = False
    mode: Literal["shadow"] = "shadow"
    model_name: str = "ViT-B-32"
    weights_path: str | None = None
    device: str = "cpu"
    batch_size: int = Field(default=8, ge=1, le=128)
    cache_path: str = "cache/visual_analysis.json"
DEFAULT_REJECTED_IMAGE_TYPES = {
    "cg": ["cover", "goods", "logo", "banner", "ui", "photo", "character_art", "announcement_art"],
    "announcement": ["logo", "banner", "ui", "photo", "goods", "cover"],
    "goods": ["logo", "banner", "ui", "photo", "game_cg", "gameplay_screenshot", "key_visual", "character_art", "cover"],
    "release": ["logo", "banner", "ui", "goods", "photo"],
    "generic": ["logo", "banner", "ui"],
    "unknown": ["logo", "banner", "ui"],
}
# Per-news image type caps. Keeping these in the config model gives older
# TOML files a safe default while allowing editorial overrides in TOML.
DEFAULT_TYPE_LIMITS = {
    "game_cg": 20,
    "gameplay_screenshot": 10,
    "key_visual": 1,
    "cover": 1,
    "character_art": 4,
    "announcement_art": 3,
    "goods": 10,
    "background_art": 20,
    "unknown": 0,
    "photo": 0,
    "logo": 0,
    "banner": 0,
    "ui": 0,
}
class ImageTypeConfig(_ConfigModel):
    banner_aspect_ratio: float = Field(default=3.0, gt=1.0)
    character_aspect_ratio: float = Field(default=1.8, gt=1.0)
    scene_aspect_ratio: float = Field(default=1.15, gt=1.0)
    minimum_gallery_group_size: int = Field(default=2, ge=2)
    minimum_type_confidence: float = Field(default=0.6, ge=0.0, le=1.0)
    unknown_requires_review: bool = True
    fallback_penalty: float = Field(default=0.25, ge=0.0, le=1.0)
    auto_select_unknown: bool = False
    max_unknown_per_news: int = Field(default=0, ge=0)
    rejected_image_types_by_requirement: dict[str, list[str]] = Field(
        default_factory=lambda: copy.deepcopy(DEFAULT_REJECTED_IMAGE_TYPES)
    )
class PrescanConfig(_ConfigModel):
    scoring: ScoringConfig
    filters: FilterConfig

```

```

network: NetworkConfig
selection: SelectionConfig
search: SearchConfig
image_types: ImageTypeConfig = Field(default_factory=ImageTypeConfig)
visual_analysis: VisualAnalysisConfig = Field(default_factory=VisualAnalysisConfig)
video: VideoConfig = Field(default_factory=VideoConfig)
def _merge(base: dict, override: dict) -> dict:
    result = copy.deepcopy(base)
    for key, value in override.items():
        if isinstance(value, dict) and isinstance(result.get(key), dict):
            result[key] = _merge(result[key], value)
        else:
            result[key] = value
    return result
def _default_path() -> Path:
    if getattr(sys, "frozen", False):
        meipass = getattr(sys, "_MEIPASS", None)
        candidates = []
        if meipass:
            candidates.append(Path(meipass) / "config" / "default.toml")
        executable = getattr(sys, "executable", None)
        if executable:
            candidates.append(Path(executable).resolve().parent / "config" / "default.toml")
    for candidate in candidates:
        if candidate.is_file():
            return candidate
    return Path(__file__).resolve().parents[2] / "config" / "default.toml"
def load_config(path: Path | str | None = None) -> PrescanConfig:
    """Load repository defaults and recursively overlay an optional TOML file."""
    default_path = _default_path()
    if not default_path.is_file():
        raise FileNotFoundError(f"default configuration is missing: {default_path}")
    try:
        with default_path.open("rb") as handle:
            payload = tomllib.load(handle)
    except tomllib.TOMLDecodeError as exc:
        raise ValueError(f"invalid default configuration: {exc}") from exc
    if path is not None:
        selected = Path(path)
        if not selected.is_file():
            raise FileNotFoundError(f"configuration is missing: {selected}")
        try:
            with selected.open("rb") as handle:
                payload = _merge(payload, tomllib.load(handle))
        except tomllib.TOMLDecodeError as exc:
            raise ValueError(f"invalid configuration: {exc}") from exc
    try:
        return PrescanConfig.model_validate(payload)
    except Exception as exc:
        raise ValueError(f"invalid prescan configuration: {exc}") from exc

```

```

from __future__ import annotations
import argparse
from pathlib import Path
from .application import Application
DEFAULT_OUTPUT_ROOT = Path.cwd() / "output"
def build_parser() -> argparse.ArgumentParser:
    parser = argparse.ArgumentParser(prog="weekly_images")
    sub = parser.add_subparsers(dest="command", required=True)
    run = sub.add_parser("run", help="process one DOCX issue")
    run.add_argument("input", type=Path)
    run.add_argument("--issue", required=True)
    run.add_argument("--output", type=Path)
    run.add_argument("--offline", action="store_true")
    run.add_argument("--no-videos", action="store_true", help="disable video discovery and downloading")
    run.add_argument("--config", type=Path)
    run.add_argument("--history-db", type=Path)
    run.add_argument("--llm-provider")
    run.add_argument("--llm-model")
    run.add_argument("--max-images", type=int)
    return parser
def main(argv=None) -> int:
    args = build_parser().parse_args(argv)
    if args.command == "run":
        output_dir = args.output or DEFAULT_OUTPUT_ROOT / args.issue
        Application(offline=args.offline, no_videos=args.no_videos, config_path=args.config, history_db=
args.history_db, max_images=args.max_images, llm_provider=args.llm_provider, llm_model=args.llm_model).r
un(args.input, issue_id=args.issue, output_dir=output_dir)
    return 0
"""Application orchestration for the v2 local pipeline."""
from __future__ import annotations
import os
from pathlib import Path
from urllib.parse import urlsplit
from .config import load_config
from .discovery.adapters import DirectImageAdapter, DynamicPageAdapter, OfficialHtmlAdapter, SteamAdapte
r, VideoAdapter
from .discovery.resolver import DefaultSourceResolver
from .discovery.x_api import create_x_adapter
from .domain import PipelineResult, ReviewReason, SourceType
from .ingestion import DocxDocumentParser, OpenAINewsAnalyzer, RuleBasedNewsAnalyzer
from .delivery.history import SQLiteHistoryStore
from .pipeline import CancellationToken, PipelineRunner, TaskRequest
class Application:
    def __init__(self, *, offline: bool = False, no_videos: bool = False, config_path=None, resolver=None,
parser=None, analyzer=None, history=None, history_db=None, max_images=None, llm_provider=None, llm_mo
del=None, source_transport=None, image_transport=None, video_downloader_factory=None, video_ffmpeg_detec
tor=None):
        self.offline = offline
        self.config = load_config(config_path)
        self.parser = parser or DocxDocumentParser()

```

```

        self.analyzer = analyzer or (OpenAINewsAnalyzer(provider=llm_provider, model=llm_model, api_key=
__import__("os").getenv("OPENAI_API_KEY")) if llm_provider and llm_model else RuleBasedNewsAnalyzer())
        self.history = history or (SQLiteHistoryStore(history_db) if history_db else None)
        self.resolver = resolver or DefaultSourceResolver(
            history_lookup=(self.history.sources_for if self.history else None),
            search_provider=(lambda _news: []) if offline else None,
            same_domain_depth=0 if offline else self.config.search.same_domain_depth,
            same_domain_transport=source_transport,
            search_max_results=self.config.search.max_results,
            search_timeout=self.config.network.timeout_seconds,
        )
        self.max_images = max_images
        self.source_transport = source_transport
        self.image_transport = image_transport
        self.no_videos = no_videos
        self.video_downloader_factory = video_downloader_factory
        self.video_ffmpeg_detector = video_ffmpeg_detector
def run(self, input_path: Path | str, *, issue_id: str, output_dir: Path | str) -> PipelineResult:
    def adapter_factory(news, source, config):
        path = urlsplit(source.url).path.casefold()
        if source.source_type is SourceType.VIDEO:
            return VideoAdapter()
        if ReviewReason.DYNAMIC_PAGE in source.review_reasons:
            return DynamicPageAdapter(transport=self.source_transport)
        if source.source_type is SourceType.DIRECT_IMAGE or path.endswith(
            ".jpg", ".jpeg", ".png", ".webp", ".gif", ".avif"
        ):
            return DirectImageAdapter()
        if source.source_type is SourceType.STEAM:
            return SteamAdapter(transport=self.source_transport)
        if source.source_type is SourceType.OFFICIAL_X:
            return create_x_adapter(public_transport=self.source_transport)
        return OfficialHtmlAdapter(transport=self.source_transport)
    def progress(event):
        if event.kind == "resolve_started" and event.news_index is not None:
            display = event.message.encode("ascii", "replace").decode("ascii")
            print(
                f"[weekly_images] resolving {event.news_index}/{event.total_news}: {display}",
                flush=True,
            )
        elif event.kind == "resolve_completed" and event.news_index is not None:
            print(
                f"[weekly_images] resolved {event.news_index}/{event.total_news}: {event.message}",
                flush=True,
            )
    runner = PipelineRunner(
        parser=self.parser,
        analyzer=self.analyzer,
        resolver=self.resolver,
        history=self.history,

```

```

        config=self.config,
        source_transport=self.source_transport,
        image_transport=self.image_transport,
        adapter_factory=adapter_factory,
        video_downloader_factory=self.video_downloader_factory,
        video_ffmpeg_detector=self.video_ffmpeg_detector,
        _legacy_output=True,
    )
    task = TaskRequest(
        input_path=input_path,
        issue_id=issue_id,
        output_dir=output_dir,
        offline=self.offline,
        no_videos=self.no_videos,
        max_images=self.max_images,
        llm_provider=None,
        llm_model=None,
        task_id=f"legacy-{issue_id}",
    )
    result = runner.run(task, progress, CancellationToken())
    pipeline_result = result.pipeline_result
    if self.no_videos:
        pipeline_result.videos = []
    return pipeline_result
from .allocator import ImageAllocator
from .dedup import Deduplicator
from .ranker import ImageRanker
from .validation import ImageValidator, placeholder_asset_reason
from .entity_matching import EntityMatchResult, EntityMatcher
from .source_policy import SourceTrustPolicy, SourceTrustResult
from .curator import ImageCurator
from .download import ImageDownloader
from .image_tying import ImageRequirementPolicy, ImageTypeClassifier, ImageTypeDecision, ImageTypeResult
__all__ = ["ImageAllocator", "Deduplicator", "ImageRanker", "ImageValidator", "placeholder_asset_reason",
    "ImageCurator", "ImageDownloader", "ImageTypeClassifier", "ImageTypeResult", "ImageTypeDecision", "ImageRequirementPolicy", "EntityMatcher", "EntityMatchResult", "SourceTrustPolicy", "SourceTrustResult"]
from __future__ import annotations
from collections import Counter, defaultdict
from collections.abc import Mapping
from typing import Iterable
from ..domain import ImageCandidate, ImageType, NewsItem
class CandidateSelection(list[ImageCandidate]):
    def __init__(self, values=(), selection_shortfall: int = 0):
        super().__init__(values)
        self.selection_shortfall = selection_shortfall
class ImageAllocator:
    def __init__(self, *, min_images: int = 5, max_images: int | None = 20, per_news_max: int = 20, minimum_score: float = 0.0, max_unknown_per_news: int | None = None, type_limits: Mapping[str, int] | None = None):

```

```

        self.min_images, self.max_images, self.per_news_max, self.minimum_score, self.max_unknown_per_news
ws = min_images, max_images, per_news_max, minimum_score, max_unknown_per_news
        # ``None`` preserves the historical direct-allocator behaviour. The
        # application always passes the configured limits.
        self.type_limits = dict(type_limits) if type_limits is not None else None
    def _type_limit(self, candidate: ImageCandidate) -> int | None:
        if candidate.signals.get("x_api_photo") is True or candidate.signals.get("socialdata_photo") is
True:
            return None
        if candidate.image_type is ImageType.UNKNOWN and self.max_unknown_per_news is not None:
            return self.max_unknown_per_news
        if self.type_limits is None:
            return None
        return self.type_limits.get(candidate.image_type.value)
    def allocate(self, news_items: Iterable[NewsItem], candidates: Iterable[ImageCandidate]) -> Candidat
eSelection:
        items = list(news_items)
        all_candidates = list(candidates)
        groups: dict[str, list[ImageCandidate]] = defaultdict(list)
        for candidate in all_candidates:
            candidate.selected = False
            if candidate.signals.get("auto_select", True) is False:
                continue
            type_limit = self._type_limit(candidate)
            if type_limit is not None and type_limit <= 0:
                continue
            if candidate.score is None or candidate.score.total >= self.minimum_score:
                groups[candidate.news_id].append(candidate)
        for group in groups.values(): group.sort(key=lambda c: (
            not (c.signals.get("x_api_photo") is True or c.signals.get("socialdata_photo") is True),
            -(c.score.total if c.score else 0),
            -(c.score.source_trust if c.score else 0),
            -((c.width or 0) * (c.height or 0)),
            c.id or "",
        ))
        for news_id, group in list(groups.items()):
            preferred = [candidate for candidate in group if candidate.signals.get("fallback_only") is n
ot True]
            if preferred:
                groups[news_id] = preferred
        selected: list[ImageCandidate] = []
        selected_counts: dict[str, Counter[str]] = defaultdict(Counter)
        def can_select(candidate: ImageCandidate) -> bool:
            if sum(selected_counts[candidate.news_id].values()) >= self.per_news_max:
                return False
            limit = self._type_limit(candidate)
            return limit is None or selected_counts[candidate.news_id][candidate.image_type.value] < lim
it
        def select(candidate: ImageCandidate) -> None:
            candidate.selected = True

```

```

        selected.append(candidate)
        selected_counts[candidate.news_id][candidate.image_type.value] += 1
    # First pass guarantees visual representation for highest-quality news.
    for item in sorted(items, key=lambda value: (-value.importance, value.id or "")):
        group = groups.get(item.id or "", [])
        if group:
            first = next((candidate for candidate in group if can_select(candidate)), None)
            if first is not None:
                select(first)
    # Second pass fills remaining slots while honoring per-news caps.
    for item in sorted(items, key=lambda value: (-value.importance, value.id or "")):
        for candidate in groups.get(item.id or "", []):
            if self.max_images is not None and len(selected) >= self.max_images: break
            if not candidate.selected and can_select(candidate):
                select(candidate)
    shortfall = sum(max(0, self.min_images - sum(candidate.selected for candidate in groups.get(item
.id or "", []))) for item in items)
    return CandidateSelection(all_candidates, selection_shortfall=shortfall)
from __future__ import annotations
from collections.abc import Iterable
from urllib.parse import urlsplit, urlunsplit
from ..config import PrescanConfig, load_config
from ..domain import CurationResult, FailureRecord, FailureStage, ImageCandidate, ImageCurationStatus, I
mageType, Issue, ReviewReason
from .allocator import ImageAllocator
from .dedup import Deduplicator
from .entity_matching import EntityMatcher
from .image_typing import ImageRequirementPolicy, ImageTypeClassifier, ImageTypeDecision, ImageTypeResul
t, _official_news_linked_gallery
from .gallery_evidence import inherit_gallery_evidence, reset_selection
from .ranker import ImageRanker
from .source_policy import SourceTrustPolicy
from .visual_analysis import VisualAnalysisService, create_local_openclip_analyzer
class ImageCurator:
    """Coordinate image typing, policy filtering, deduplication, ranking and allocation."""
    def __init__(self, config: PrescanConfig | None = None, *, visual_analyzer=None):
        self.config = config or load_config()
        self.visual_service = None
        self.visual_initialization_error = None
        if self.config.visual_analysis.enabled:
            try:
                analyzer = visual_analyzer or create_local_openclip_analyzer(
                    self.config.visual_analysis.model_name,
                    self.config.visual_analysis.weights_path or "",
                    self.config.visual_analysis.device,
                )
                self.visual_service = VisualAnalysisService(
                    analyzer,
                    cache_path=self.config.visual_analysis.cache_path,
                    batch_size=self.config.visual_analysis.batch_size,

```



```

        )
        except Exception as exc:
            # Optional shadow analysis must never stop the rule-based path.
            self.visual_initialization_error = str(exc)

    @staticmethod
    def _source_key(value: str) -> str:
        parts = urlsplit(value.strip())
        return urlunsplit((parts.scheme.casefold(), parts.netloc.casefold(), parts.path.rstrip("/"), parts.query, ""))

    @classmethod
    def _source_linked(cls, item, candidate: ImageCandidate) -> bool:
        if not item.source_urls:
            return False
        roots = {cls._source_key(url) for url in item.source_urls if url}
        candidate_sources = [candidate.source_url, candidate.news_source_url or "", candidate.signals.get("root_source_url", "")]
        alternates = candidate.signals.get("source_chain_alternates", "")
        if isinstance(alternates, str):
            candidate_sources.extend(alternates.split("|"))
        return any(cls._source_key(str(url)) in roots for url in candidate_sources if url)

    def curate(self, issue: Issue, candidates: Iterable[ImageCandidate], history=None) -> CurationResult:
        values = list(candidates)
        for candidate in values:
            reset_selection(candidate)
        failures: list[FailureRecord] = []
        if self.config.visual_analysis.enabled:
            if self.visual_initialization_error:
                failures.append(FailureRecord(
                    stage=FailureStage.CURATE,
                    code="visual_shadow_unavailable",
                    message=self.visual_initialization_error,
                    retryable=False,
                ))
            elif self.visual_service is not None:
                result = self.visual_service.annotate(values)
                if result.failed:
                    failures.append(FailureRecord(
                        stage=FailureStage.CURATE,
                        code="visual_shadow_inference_failed",
                        message=f"local visual shadow inference failed for {result.failed} image(s)",
                        retryable=True,
                    ))
        historical_hashes = {}
        if history is not None and hasattr(history, "known_image"):
            for candidate in values:
                try:
                    if history.known_image(candidate.sha256, candidate.perceptual_hash) is not None:
                        if ReviewReason.HISTORICAL_DUPLICATE not in candidate.review_reasons:
                            candidate.review_reasons.append(ReviewReason.HISTORICAL_DUPLICATE)

```

```

        except Exception as exc:
            failures.append(FailureRecord(stage=FailureStage.HISTORY, news_id=candidate.news_id,
candidate_id=candidate.id, code="history_lookup_error", message=str(exc), retryable=True))
        by_news = {item.id: item for item in issue.news_items}
        classifier = ImageTypeClassifier(self.config.image_types)
        policy = ImageRequirementPolicy(self.config.image_types)
        entity_matcher = EntityMatcher()
        inherit_gallery_evidence(by_news, values, _official_news_linked_gallery, classifier,
                                lambda news, candidate: entity_matcher.match(news, candidate).matched i
s True)
        source_policy = SourceTrustPolicy()
        eligible: list[ImageCandidate] = []
        filtered: list[ImageCandidate] = []
        for candidate in values:
            item = by_news.get(candidate.news_id)
            x_api_photo = (
                candidate.signals.get("x_api_photo") is True
                or candidate.signals.get("socialdata_photo") is True
            )
            priority_x_photo = False
            candidate.selection_reasons.clear()
            try:
                if item is None:
                    typed = ImageTypeResult(ImageType.UNKNOWN, 0.0, ("news_not_found",))
                    decision = ImageTypeDecision(True, requires_review=True, auto_select=False, type_mat
ch=0.1)
                else:
                    typed = classifier.classify(item, candidate)
                    decision = policy.evaluate(item, typed)
                    candidate.image_type = typed.image_type
                    candidate.image_type_confidence = typed.confidence
                    if not decision.auto_select and not candidate.signals.get("invalid_reason"):
                        candidate.selection_reasons.append("type_evidence_insufficient" if typed.confidence
< 0.8 else "automatic_type_selection_disabled")
                    source_result = source_policy.classify(candidate)
                    candidate.signals["source_tier"] = source_result.tier
                    candidate.signals["source_officiality"] = source_result.officiality
                    candidate.signals["source_is_official"] = source_result.is_official
                    source_linked = item is not None and self._source_linked(item, candidate)
                    candidate.signals["source_linked"] = source_linked
                    priority_x_photo = x_api_photo and source_linked
                    if priority_x_photo:
                        candidate.selection_reasons.append("socialdata_x_photo_priority")
                    if not source_linked:
                        candidate.selection_reasons.append("source_not_linked_to_news")
                    if ReviewReason.UNCERTAIN_MATCH not in candidate.review_reasons:
                        candidate.review_reasons.append(ReviewReason.UNCERTAIN_MATCH)
                    failures.append(FailureRecord(stage=FailureStage.CURATE, news_id=candidate.news_id,
candidate_id=candidate.id, code="news_source_unlinked", message="candidate source is not linked to this
news item's source URLs", source_url=candidate.source_url, retryable=False))

```

```

        if typed.supporting_signals:
            candidate.signals["image_type_supporting_signals"] = ",".join(typed.supporting_signals)

    candidate.signals["type_match"] = decision.type_match
    width, height = candidate.width or 0, candidate.height or 0
    quality_eligible = (
        width >= self.config.filters.min_width
        and height >= self.config.filters.min_height
        and width * height >= self.config.filters.min_pixels
    )
    candidate.signals["quality_eligible"] = quality_eligible
    animated_source = candidate.animated_source or candidate.signals.get("animated_source")

    if animated_source:
        candidate.animated_source = True
        candidate.selection_reasons.append("animated_source_requires_review")
        if ReviewReason.IMAGE_TYPE_REVIEW not in candidate.review_reasons:
            candidate.review_reasons.append(ReviewReason.IMAGE_TYPE_REVIEW)
    candidate.signals["auto_select"] = (
        (priority_x_photo or (decision.auto_select and quality_eligible))
        and source_linked
        and not animated_source
    )

    if not quality_eligible:
        candidate.selection_reasons.append("below_minimum_resolution")
        if ReviewReason.IMAGE_QUALITY_REVIEW not in candidate.review_reasons:
            candidate.review_reasons.append(ReviewReason.IMAGE_QUALITY_REVIEW)
        if not priority_x_photo:
            failures.append(FailureRecord(stage=FailureStage.CURATE, news_id=candidate.news_id, candidate_id=candidate.id, code="image_quality_below_auto_select_threshold", message="image is retained for review but does not meet automatic selection dimensions", source_url=candidate.image_url, retryable=False))

    if candidate.signals.get("invalid_reason"):
        candidate.curation_status = ImageCurationStatus.INVALID
        candidate.signals["auto_select"] = False
        candidate.selection_reasons.append(f"invalid:{candidate.signals['invalid_reason']}")
    if candidate.signals.get("conversion_error"):
        candidate.signals["auto_select"] = False
        candidate.selection_reasons.append("review_image_conversion_failed")
    if priority_x_photo:
        candidate.signals.pop("fallback_only", None)
    elif decision.fallback_only:
        candidate.signals["fallback_only"] = True
    if decision.requires_review and ReviewReason.IMAGE_TYPE_REVIEW not in candidate.review_reasons:
        candidate.review_reasons.append(ReviewReason.IMAGE_TYPE_REVIEW)

    try:
        # Remove the previous pass's derived conflict before
        # recomputing entity evidence for replay/review workflows.
        candidate.signals.pop("entity_conflict", None)

```

```

        match = entity_matcher.match(item, candidate) if item is not None else None
    except Exception as exc:
        # A malformed page-local signal must not discard the
        # other candidates for this news item.
        candidate.signals["entity_match"] = "unknown"
        candidate.signals["entity_match_confidence"] = 0.0
        candidate.signals["auto_select"] = False
        if ReviewReason.UNCERTAIN_MATCH not in candidate.review_reasons:
            candidate.review_reasons.append(ReviewReason.UNCERTAIN_MATCH)
        eligible.append(candidate)
        failures.append(FailureRecord(stage=FailureStage.CURATE, news_id=candidate.news_id,
candidate_id=candidate.id, code="entity_match_error", message=str(exc), source_url=candidate.image_url,
retryable=False))
        continue
    if match is not None:
        candidate.signals["entity_match"] = match.matched if match.matched is not None else
"unknown"

        candidate.signals["entity_match_confidence"] = match.confidence
        candidate.signals["entity_match_evidence"] = ",".join(match.supporting_signals)
        candidate.signals["official_domain_match"] = match.official_domain_match
        if match.conflicting_entities:
            candidate.signals["entity_conflict"] = ",".join(match.conflicting_entities)
        if match.matched is False:
            if priority_x_photo:
                candidate.selection_reasons.append("socialdata_x_photo_source_overrides_ent
ty_classifier")

            else:
                candidate.selected = False
                candidate.selection_reasons.append("entity_conflict")
                filtered.append(candidate)
                failures.append(FailureRecord(stage=FailureStage.CURATE, news_id=candidate.n
ews_id, candidate_id=candidate.id, code="entity_mismatch", message="candidate conflicts with the news ga
me entity", source_url=candidate.image_url, retryable=False))
                continue
            if match.matched is None:
                candidate.selection_reasons.append("entity_unverified")
                if not priority_x_photo:
                    candidate.signals["auto_select"] = False
                    if ReviewReason.UNCERTAIN_MATCH not in candidate.review_reasons:
                        candidate.review_reasons.append(ReviewReason.UNCERTAIN_MATCH)
                    failures.append(FailureRecord(stage=FailureStage.CURATE, news_id=candidate.n
ews_id, candidate_id=candidate.id, code="entity_unverified", message="candidate lacks sufficient game/en
tity context", source_url=candidate.image_url, retryable=False))
                elif not priority_x_photo and (
                    (not source_result.is_official and match.confidence < 0.8)
                    or (
                        source_result.tier in {"official_brand_page", "official_store"}
                        and not any(signal in match.supporting_signals for signal in ("entity_name_i
n_context", "page_context", "steam_app_id_match", "explicit_entity_signal"))
                    )
                )

```

```

        ):
            candidate.signals["auto_select"] = False
            if ReviewReason.UNCERTAIN_MATCH not in candidate.review_reasons:
                candidate.review_reasons.append(ReviewReason.UNCERTAIN_MATCH)
            failures.append(FailureRecord(stage=FailureStage.CURATE, news_id=candidate.news_id,
            candidate_id=candidate.id, code="entity_unverified", message="non-official candidate requires stronger entity evidence", source_url=candidate.image_url, retryable=False))
            if decision.accepted or priority_x_photo:
                eligible.append(candidate)
            else:
                candidate.selected = False
                candidate.selection_reasons.append(f"image_type_rejected:{typed.image_type.value}")
                filtered.append(candidate)
                failures.append(FailureRecord(
                    stage=FailureStage.CURATE,
                    news_id=candidate.news_id,
                    candidate_id=candidate.id,
                    code="image_type_rejected",
                    message=f"image_type={typed.image_type.value}; reason={decision.rejection_reason or 'rejected by image requirement policy'}",
                    source_url=candidate.image_url,
                    retryable=False,
                ))
        except Exception as exc:
            candidate.image_type = ImageType.UNKNOWN
            candidate.image_type_confidence = 0.0
            candidate.signals["type_match"] = 0.1
            candidate.signals["auto_select"] = False
            if ReviewReason.IMAGE_TYPE_REVIEW not in candidate.review_reasons:
                candidate.review_reasons.append(ReviewReason.IMAGE_TYPE_REVIEW)
            eligible.append(candidate)
            failures.append(FailureRecord(
                stage=FailureStage.CURATE,
                news_id=candidate.news_id,
                candidate_id=candidate.id,
                code="image_type_error",
                message=str(exc),
                source_url=candidate.image_url,
                retryable=False,
            ))
    ))
    ranker = ImageRanker(self.config.scoring, self.config.image_types)
    ranked_all: list[ImageCandidate] = []
    for news_id in sorted({candidate.news_id for candidate in values}):
        item = by_news.get(news_id)
        group = [candidate for candidate in values if candidate.news_id == news_id]
        ranked_all.extend(ranker.rank(item, group) if item is not None else group)
    # Deduplication is news-local and affects only automatic selection.
    # Every valid member stays in the result for review and provenance.
    eligible_ids = {id(candidate) for candidate in eligible}
    ranked_eligible = [candidate for candidate in ranked_all if id(candidate) in eligible_ids]

```

```

all_deduped = Deduplicator().deduplicate(ranked_eligible, historical_hashes=historical_hashes)
auto_eligible_ids = {
    id(candidate) for candidate in eligible
    if candidate.signals.get("auto_select", True) is not False
}
manual_candidates = [
    candidate for candidate in eligible
    if candidate.signals.get("auto_select", True) is False
]
deduped = Deduplicator().deduplicate(
    (candidate for candidate in ranked_eligible if id(candidate) in auto_eligible_ids),
    historical_hashes=historical_hashes,
)
# Preserve duplicate relationships even when one member is not
# eligible for automatic selection.
for candidate in all_deduped.duplicates:
    if candidate not in deduped.duplicates and candidate.signals.get("duplicate_of"):
        candidate.selected = False
ranked = sorted(deduped.unique, key=lambda c: (
    not (c.signals.get("x_api_photo") is True or c.signals.get("socialdata_photo") is True),
    -(c.score.total if c.score else 0.0),
    -(c.score.source_trust if c.score else 0.0),
    -((c.width or 0) * (c.height or 0)),
    c.id or "",
))
allocated = ImageAllocator(
    min_images=self.config.selection.min_images,
    max_images=None,
    per_news_max=self.config.selection.per_news_max,
    minimum_score=self.config.selection.minimum_score,
    max_unknown_per_news=self.config.image_types.max_unknown_per_news,
    type_limits=self.config.selection.type_limits,
).allocate(issue.news_items, ranked)
selected_ids = {id(candidate) for candidate in allocated if candidate.selected}
for candidate in [*allocated, *deduped.duplicates, *manual_candidates, *filtered]:
    if candidate.curation_status is ImageCurationStatus.INVALID:
        continue
    if candidate.signals.get("duplicate_of"):
        if "duplicate_of_better_candidate" not in candidate.selection_reasons:
            candidate.selection_reasons.append("duplicate_of_better_candidate")
    elif id(candidate) in selected_ids:
        candidate.selection_reasons.append("selected_by_rank_and_policy")
    elif not candidate.selection_reasons:
        reason = "news_selection_limit" if sum(c.selected for c in allocated if c.news_id == candidate.news_id) >= self.config.selection.per_news_max else "type_limit_or_minimum_score"
        candidate.selection_reasons.append(reason)
    result_candidates = [*allocated, *deduped.duplicates, *manual_candidates]
    sort_key = lambda c: (c.news_id, not (c.signals.get("x_api_photo") is True or c.signals.get("socialdata_photo") is True), -(c.score.total if c.score else 0.0), -(c.score.source_trust if c.score else 0.0), -((c.width or 0) * (c.height or 0)), c.id or "")

```

```

        result_candidates.sort(key=sort_key)
        filtered.sort(key=sort_key)
        return CurationResult(candidates=result_candidates, filtered_candidates=filtered, failures=failures, selection_shortfall=allocated.selection_shortfall)
from __future__ import annotations
from dataclasses import dataclass
from functools import lru_cache
from pathlib import Path
from typing import Iterable
from PIL import Image, ImageChops, ImageStat
from ..domain import ImageCandidate, ImageCurationStatus, ReviewReason
@dataclass
class DedupResult:
    unique: list[ImageCandidate]
    duplicates: list[ImageCandidate]
def _original_hash(candidate: ImageCandidate) -> str | None:
    return candidate.original_sha256 or candidate.sha256
@lru_cache(maxsize=128)
def _comparison_pixels(path: str, modified_ns: int, byte_size: int) -> tuple[float, Image.Image] | None:
    """Cache bounded decoded comparisons, never full-size original pixels."""
    try:
        with Image.open(Path(path)) as source:
            source.seek(0)
            aspect = source.width / source.height
            pixels = source.convert("RGB")
            pixels.thumbnail((128, 128), Image.Resampling.LANCZOS)
            return aspect, pixels.resize((128, 128), Image.Resampling.LANCZOS)
    except (OSError, ValueError):
        return None
def _pixels(candidate: ImageCandidate) -> tuple[float, Image.Image] | None:
    """Load original content; metadata invalidates cache when a file changes."""
    path = candidate.original_path or candidate.local_path
    if not path:
        return None
    try:
        resolved = Path(path).resolve()
        stat = resolved.stat()
        return _comparison_pixels(str(resolved), stat.st_mtime_ns, stat.st_size)
    except (OSError, ValueError):
        return None
def _confirmed_visual_duplicate(left: ImageCandidate, right: ImageCandidate) -> bool:
    if not left.perceptual_hash or not right.perceptual_hash:
        return False
    try:
        distance = (int(left.perceptual_hash, 16) ^ int(right.perceptual_hash, 16)).bit_count()
    except ValueError:
        return False
    if distance > 8:
        return False
    a, b = _pixels(left), _pixels(right)

```

```

    if a is None or b is None:
        return False
    # A hash collision or two similarly composed CGs must not be enough.
    # Compare normalized decoded pixels too, allowing modest codec noise.
    if abs(a[0] - b[0]) > 0.01:
        return False
    difference = ImageChops.difference(a[1], b[1])
    mean_error = sum(ImageStat.Stat(difference).mean) / 3
    if mean_error > 3.0:
        return False
    # Codec noise is diffuse. A changed expression/pose is a concentrated
    # patch and must not be merged just because most of the background agrees.
    channels = difference.split()
    maximum = ImageChops.lighter(ImageChops.lighter(channels[0], channels[1]), channels[2])
    changed_fraction = sum(maximum.histogram()[17:]) / (128 * 128)
    if changed_fraction > 0.025:
        return False
    for y in range(0, 128, 8):
        for x in range(0, 128, 8):
            tile_error = sum(ImageStat.Stat(difference.crop((x, y, x + 8, y + 8))).mean) / 3
            if tile_error > 8.0:
                return False
    return True

def _representative_key(candidate: ImageCandidate) -> tuple:
    # Detail evidence comes before rank score and encoded file size. Original
    # dimensions are used when present; byte size is deliberately ignored.
    detail = float(candidate.signals.get("visible_detail_score", candidate.signals.get("sharpness_score"
, 0.0)) or 0.0)
    width = candidate.original_width or candidate.width or 0
    height = candidate.original_height or candidate.height or 0
    return (
        not (candidate.signals.get("x_api_photo") is True or candidate.signals.get("socialdata_photo") i
s True),
        candidate.signals.get("auto_select", True) is False,
        candidate.signals.get("entity_match") is not True,
        -detail,
        -min(width, 1280) * min(height, 720),
        -(candidate.score.total if candidate.score else 0.0),
        candidate.id or "",
    )

class Deduplicator:
    def deduplicate(self, candidates: Iterable[ImageCandidate], *, historical_hashes: dict[str, str] | N
one = None) -> DedupResult:
        unique: list[ImageCandidate] = []
        duplicates: list[ImageCandidate] = []
        historical_hashes = historical_hashes or {}
        groups: dict[str, list[ImageCandidate]] = {}
        for candidate in candidates:
            groups.setdefault(candidate.news_id, []).append(candidate)
        for group in groups.values():

```



```

        representatives: list[ImageCandidate] = []
        for candidate in sorted(group, key=_representative_key):
            if candidate.signals.get("fallback_old_material") and ReviewReason.FALLBACK_OLD_MATERIAL
            not in candidate.review_reasons:
                candidate.review_reasons.append(ReviewReason.FALLBACK_OLD_MATERIAL)
            sha = _original_hash(candidate)
            historical = (sha and sha in historical_hashes) or (candidate.perceptual_hash and candi
            ate.perceptual_hash in historical_hashes.values())
            if historical and ReviewReason.HISTORICAL_DUPLICATE not in candidate.review_reasons:
                candidate.review_reasons.append(ReviewReason.HISTORICAL_DUPLICATE)
            duplicate_of = next((rep for rep in representatives if sha and sha == _original_hash(rep
            )), None)
            duplicate_kind = "sha256" if duplicate_of is not None else None
            if duplicate_of is None:
                duplicate_of = next((rep for rep in representatives if _confirmed_visual_duplicate(c
            andidate, rep)), None)
            if duplicate_of is not None:
                duplicate_kind = "perceptual_pixels"
            if duplicate_of is not None:
                candidate.selected = False
                candidate.curation_status = ImageCurationStatus.UNSELECTED
                candidate.signals["duplicate_of"] = str(duplicate_of.id or "")
                candidate.signals["duplicate_kind"] = duplicate_kind or "sha256"
                candidate.signals["duplicate_reason"] = "same_news_same_scene"
                if "duplicate_of_better_candidate" not in candidate.selection_reasons:
                    candidate.selection_reasons.append("duplicate_of_better_candidate")
                duplicates.append(candidate)
                continue
            possible = next((rep for rep in representatives if candidate.perceptual_hash and candida
            te.perceptual_hash == rep.perceptual_hash), None)
            if possible is not None:
                candidate.signals["possible_duplicate_of"] = str(possible.id or "")
            else:
                candidate.signals.pop("possible_duplicate_of", None)
                candidate.signals.pop("duplicate_of", None)
                candidate.signals.pop("duplicate_kind", None)
                candidate.signals.pop("duplicate_reason", None)
                representatives.append(candidate)
                unique.append(candidate)
        return DedupResult(unique=unique, duplicates=duplicates)
    """Download candidates into a temporary staging area before ranking."""
    from __future__ import annotations
    import asyncio
    import hashlib
    import os
    import uuid
    import time
    from threading import Lock
    from concurrent.futures import ThreadPoolExecutor
    from collections.abc import Iterable

```

```

from io import BytesIO
from pathlib import Path
from typing import Any, Callable
from urllib.parse import urlsplit, urlunsplit
from PIL import Image, ImageFilter
from ..config import PrescanConfig
from ..domain import FailureRecord, FailureStage, ImageCandidate
from ..discovery.http import SafeHttpClient
from ..validation import ImageValidator, meaningless_asset_reason, placeholder_asset_reason
_EXTENSIONS = {
    "image/jpeg": ".jpg",
    "image/png": ".png",
    "image/gif": ".gif",
    "image/webp": ".webp",
    "image/avif": ".avif",
}
def _perceptual_hash(data: bytes) -> str | None:
    try:
        with Image.open(BytesIO(data)) as image:
            grayscale = image.convert("L").resize((8, 8), Image.Resampling.LANCZOS)
            values = list(grayscale.get_flattened_data() if hasattr(grayscale, "get_flattened_data") else grayscale.getdata())
            average = sum(values) / len(values)
            bits = "".join("1" if value >= average else "0" for value in values)
            return f"{int(bits, 2):016x}"
    except Exception:
        return None
def _image_quality_signals(data: bytes, width: int, height: int) -> dict[str, float | bool | str]:
    """Measure source detail, blur, and blockiness without using byte size."""
    try:
        with Image.open(BytesIO(data)) as source:
            frame_count = int(getattr(source, "n_frames", 1) or 1)
            has_alpha = "A" in source.getbands() or "transparency" in source.info
            frame = source.copy().convert("L")
            frame.thumbnail((384, 384), Image.Resampling.LANCZOS)
            edge_histogram = frame.filter(ImageFilter.FIND_EDGES).histogram()
            count = max(1, sum(edge_histogram))
            edge_energy = sum(index * amount for index, amount in enumerate(edge_histogram)) / count / 2
55.0
            pixels = frame.load()
            frame_width, frame_height = frame.size
            second_derivatives: list[int] = []
            boundary_diffs: list[int] = []
            non_boundary_diffs: list[int] = []
            for y in range(1, frame_height - 1):
                for x in range(1, frame_width - 1):
                    center = pixels[x, y]
                    laplace = abs(4 * center - pixels[x - 1, y] - pixels[x + 1, y] - pixels[x, y - 1] -
pixels[x, y + 1])
                    second_derivatives.append(laplace)

```

```

        horizontal = abs(center - pixels[x - 1, y])
        vertical = abs(center - pixels[x, y - 1])
        if x % 8 == 0:
            boundary_diffs.append(horizontal)
        elif x % 8 == 1:
            non_boundary_diffs.append(horizontal)
        if y % 8 == 0:
            boundary_diffs.append(vertical)
        elif y % 8 == 1:
            non_boundary_diffs.append(vertical)
    laplace_energy = sum(second_derivatives) / max(1, len(second_derivatives)) / 1020.0
    blockiness = max(0.0, min(1.0, (
        sum(boundary_diffs) / max(1, len(boundary_diffs))
        - sum(non_boundary_diffs) / max(1, len(non_boundary_diffs))
    ) / 255.0 * 4.0))
    # These are comparative ranking signals, not calibrated quality
    # labels. Preserve raw pixels and validate thresholds offline.
    sharpness = max(0.0, min(1.0, 0.55 * edge_energy * 2.8 + 0.45 * laplace_energy * 8.0))
    resolution = max(0.0, min(1.0, min(width / 1280.0, height / 720.0)))
    detail = max(0.0, min(1.0, 0.65 * resolution + 0.35 * sharpness - 0.15 * blockiness))
    return {
        "sharpness_score": round(sharpness, 6),
        "visible_detail_score": round(detail, 6),
        "blur_score": round(max(0.0, min(1.0, 1.0 - laplace_energy * 8.0)), 6),
        "blockiness_score": round(blockiness, 6),
        "transparent": has_alpha,
        "animated_source": frame_count > 1,
        "animation_frame_count": frame_count,
    }
except Exception:
    return {"quality_analysis_failed": True}
class ImageDownloader:
    def __init__(self, config: PrescanConfig, *, transport: Callable[..., Any] | None = None):
        self.processing_seconds = 0.0
        self._timing_lock = Lock()
        self.client = SafeHttpClient(
            transport=transport,
            timeout=config.network.timeout_seconds,
            max_retries=config.network.max_retries,
            max_response_bytes=config.filters.max_image_bytes,
            user_agent=config.network.user_agent,
        )
        self.validator = ImageValidator(
            min_width=config.filters.min_width,
            min_height=config.filters.min_height,
            min_pixels=config.filters.min_pixels,
            max_bytes=config.filters.max_image_bytes,
        )
    def _prepare(self, candidates, directory):
        root = Path(directory)

```

```

root.mkdir(parents=True, exist_ok=True)
seen_urls: set[tuple[str, str]] = set()
candidate_order: list[str] = []
duplicate_urls: dict[tuple[str, str], ImageCandidate] = {}
pending: list[ImageCandidate] = []
filtered: list[FailureRecord] = []
invalid: list[ImageCandidate] = []
for candidate in candidates:
    parts = urlsplit(candidate.image_url.strip())
    normalized = urlunsplit((parts.scheme.casefold(), parts.netloc.casefold(), parts.path, parts
.query, ""))
    key = (candidate.news_id, normalized)
    if key in seen_urls:
        original = duplicate_urls[key]
        alternate_roots = [str(original.signals.get("root_source_url", "")), str(candidate.signa
ls.get("root_source_url", ""))]
        roots = list(dict.fromkeys(value for value in alternate_roots if value))
        if roots:
            original.signals["source_chain_alternates"] = "|".join(roots)
            continue
        seen_urls.add(key)
        duplicate_urls[key] = candidate
        candidate_order.append(str(candidate.id))
        placeholder_reason = placeholder_asset_reason(candidate)
        if placeholder_reason:
            candidate.downloadable = False
            candidate.signals["invalid_reason"] = placeholder_reason
            invalid.append(candidate)
            filtered.append(self._failure(candidate, placeholder_reason, "placeholder asset was reje
cted before download", False))
            continue
        reason = meaningless_asset_reason(candidate)
        if reason or parts.scheme not in {"http", "https"}:
            failure_code = "filtered_invalid_material"
            candidate.downloadable = False
            candidate.signals["invalid_reason"] = reason or "invalid_scheme"
            invalid.append(candidate)
            filtered.append(self._failure(candidate, failure_code, reason or "invalid_scheme", False
))
            continue
        pending.append(candidate)
    return root, candidate_order, pending, filtered, invalid
def _process_response(self, candidate, response, root, temporary=None):
    started = time.monotonic()
    try:
        status = int(getattr(response, "status_code", 200))
        if status < 200 or status >= 300:
            return candidate, self._failure(candidate, "http_error", f"image request returned HTTP {
status}"), status >= 500 or status in {408, 429}), None
        final_response_url = str(getattr(response, "url", candidate.image_url) or candidate.image_ur

```

```

1)
    redirected = candidate.model_copy(update={"image_url": final_response_url})
    if placeholder_asset_reason(redirected):
        return candidate, self._failure(candidate, "placeholder_image", "redirected URL is a placeholder asset", False), None
    data = temporary.read_bytes() if temporary is not None else (getattr(response, "content", b"")) or b""
    declared_mime = getattr(response, "headers", {}).get("content-type")
    validation = self.validator.validate(data, declared_mime)
    if not validation.valid:
        return candidate, self._failure(candidate, validation.reason or "invalid_image", "downloaded image failed validation", False), None
    suffix = _EXTENSIONS.get(validation.mime_type or "", ".img")
    target = root / f"{candidate.id}{suffix}"
    if temporary is None:
        temporary = root / f"{candidate.id}-{uuid.uuid4().hex}.part"
        temporary.write_bytes(data)
    os.replace(temporary, target)
    candidate.width, candidate.height = validation.width, validation.height
    candidate.mime_type, candidate.byte_size = validation.mime_type, len(data)
    candidate.sha256, candidate.perceptual_hash = hashlib.sha256(data).hexdigest(), _perceptual_hash(data)
    candidate.local_path, candidate.downloadable = str(target), True
    candidate.original_path = str(target)
    candidate.original_mime_type = validation.mime_type
    candidate.original_byte_size = len(data)
    candidate.original_sha256 = candidate.sha256
    candidate.original_width, candidate.original_height = validation.width, validation.height
    candidate.signals.update(_image_quality_signals(data, validation.width or 0, validation.height or 0))
    return candidate, None, final_response_url
except Exception as exc:
    return candidate, self._failure(candidate, "download_error", str(exc), True), None
finally:
    if temporary is not None:
        temporary.unlink(missing_ok=True)
    with self._timing_lock:
        self.processing_seconds += time.monotonic() - started
def _finish(self, outcomes, candidate_order, filtered, invalid):
    accepted: list[ImageCandidate] = []
    failures: list[FailureRecord] = list(filtered)
    for candidate, failure, final_url in outcomes:
        if failure is not None:
            if candidate is not None:
                candidate.downloadable = False
                candidate.local_path = None
                candidate.signals["invalid_reason"] = failure.code
                invalid.append(candidate)
            failures.append(failure)
    continue

```

```

        if candidate is None:
            continue
        accepted.append(candidate)
    # Keep invalid and downloaded candidate records. Failures are attached
    # separately for diagnostics; callers can still account for every URL.
    by_id = {str(candidate.id): candidate for candidate in [*accepted, *invalid]}
    all_candidates = [by_id[candidate_id] for candidate_id in candidate_order if candidate_id in by_
id]

    return all_candidates, failures

def download(self, candidates, directory):
    root, order, pending, filtered, invalid = self._prepare(candidates, directory)
    def fetch(candidate):
        try:
            response = self.client.get(candidate.image_url)
            return self._process_response(candidate, response, root)
        except Exception as exc:
            return candidate, self._failure(candidate, "download_error", str(exc), True), None
    with ThreadPoolExecutor(max_workers=min(8, max(4, len(pending)))) as pool:
        outcomes = list(pool.map(fetch, pending))
    return self._finish(outcomes, order, filtered, invalid)

async def download_async(self, candidates, directory, *, client, executor, concurrency=8):
    """Stream with a bounded window; CPU processing never runs on the loop."""
    root, order, pending, filtered, invalid = self._prepare(candidates, directory)
    loop = asyncio.get_running_loop()
    outcomes = [None] * len(pending)
    queue = asyncio.Queue()
    for index, candidate in enumerate(pending):
        queue.put_nowait((index, candidate))
    async def worker():
        while not queue.empty():
            index, candidate = queue.get_nowait()
            temporary = root / f".{candidate.id}-{uuid.uuid4().hex}.part"
            try:
                response = await client.download(candidate.image_url, temporary)
                # At most concurrency downloaded files can wait for the
                # limited processing executor, including files in flight.
                processing = loop.run_in_executor(executor, self._process_response, candidate, respo
nse, root, temporary)
            try:
                outcomes[index] = await asyncio.shield(processing)
            except asyncio.CancelledError:
                await processing
                raise
        except Exception as exc:
            outcomes[index] = candidate, self._failure(candidate, "download_error", str(exc), Tr
ue), None
    finally:
        temporary.unlink(missing_ok=True)
        queue.task_done()
    tasks = [asyncio.create_task(worker()) for _ in range(min(concurrency, len(pending)))]

```

```

        if self.exporter.session is self.session:
            self.operation_status.setText("正在导出, 请等待完成后修改审核决定")
            return
        for candidate in self._selected_items():
            self.session.set_decision(str(candidate.id), decision)
            self.decision_changed.emit(str(candidate.id), decision.value)
        self.refresh()
    def _selected_items(self) -> list[Any]:
        rows = sorted({index.row() for index in self.media_list.selectedIndexes()})
        if not rows and self._selected() is not None:
            return [self._selected()]
        return [self._items[row] for row in rows if 0 <= row < len(self._items)]
    def accept_selected(self) -> None:
        self._set_selected_decision(ReviewDecision.ACCEPTED)
    def reject_selected(self) -> None:
        self._set_selected_decision(ReviewDecision.REJECTED)
    def pending_selected(self) -> None:
        self._set_selected_decision(ReviewDecision.PENDING)
    def _source_url(self, candidate: Any) -> str:
        return str(getattr(candidate, "news_source_url", None) or getattr(candidate, "parent_source_url", None)
        , None)
        or getattr(candidate, "source_url", None) or getattr(candidate, "image_url", None) or
        "")
    def _official_url(self, candidate: Any) -> str:
        news = self._news.get(str(candidate.news_id), {})
        official = news.get("official_url") or news.get("official_site_url")
        if official:
            return str(official)
        if code(getattr(candidate, "source_type", "")) in {"official_site", "steam"}:
            return str(getattr(candidate, "source_url", ""))
        return ""
    def open_source(self) -> None:
        candidate = self._selected()
        if candidate and self._source_url(candidate):
            QDesktopServices.openUrl(QUrl(self._source_url(candidate)))
    def open_official(self) -> None:
        candidate = self._selected()
        if candidate and self._official_url(candidate):
            QDesktopServices.openUrl(QUrl(self._official_url(candidate)))
    def open_original(self) -> None:
        candidate = self._selected()
        if candidate and getattr(candidate, "image_url", None):
            QDesktopServices.openUrl(QUrl(str(candidate.image_url)))
    def play_video(self) -> None:
        candidate = self._selected()
        path = self.session._source_for(candidate) if self.session and candidate else None
        if path:
            QDesktopServices.openUrl(QUrl.fromLocalFile(str(path)))
    def retry_selected(self) -> None:
        if self.exporter.session is self.session and self.session is not None:

```

```

        self.operation_status.setText("正在导出, 请等待完成后重试")
        return
    candidate = self._selected()
    news_id = candidate.news_id if candidate else self._news_filter()
    if not news_id:
        return
    self._bind_retry_context(str(news_id))
    if not self.retry_url() and candidate:
        self.retry_url_edit.setText(self._official_url(candidate))
    elif not self.retry_url():
        news = self._news.get(str(news_id), {})
        self.retry_url_edit.setText(str(news.get("official_url") or news.get("official_site_url") or
        ""))

    url = QUrl(self.retry_url())
    if url.scheme().casefold() not in {"http", "https"} or not url.host():
        self.operation_status.setText("请输入可访问的官网地址 (http 或 https), 再重试本条新闻")
        self.retry_url_edit.setFocus()
        return
    self.operation_status.setText("正在重试本条新闻...")
    self.retry_requested.emit(str(news_id))
def _bind_retry_context(self, news_id: str | None) -> None:
    context = (id(self.session), str(news_id or ""))
    if context != self._retry_context:
        self.retry_url_edit.clear()
        self.operation_status.clear()
        self._retry_context = context
def show_retry_result(self, success: bool) -> None:
    self.operation_status.setText("重试完成, 候选已更新" if success else "重试未完成, 请检查官网地址
与设置后重试")
def show_retry_failure(self, *args: Any) -> None:
    self.operation_status.setText("重试失败, 请展开技术详情查看原因")
    self.raw_details.setPlainText("\n".join(str(value) for value in args))
    self.raw_toggle.setChecked(True)
def retry_url(self) -> str:
    return self.retry_url_edit.text().strip()
def export_final(self) -> None:
    if self.session is None or not self.exporter.start(self.session):
        return
    self._export_requested_count = len(self.session.accepted_images()) + len(self.session.accepted_v
ideos())
    self.export_button.setEnabled(False)
    self.export_status.setText("正在导出已选媒体...")
    for button in (self.accept_button, self.reject_button, self.pending_button, self.retry_button):
        button.setEnabled(False)
def _export_done(self, session: ReviewSession, manifest: dict | None, error: str) -> None:
    self.export_button.setEnabled(self.session is not None)
    if session is not self.session:
        return
    if manifest is not None:
        exported = int(manifest.get("image_count", 0)) + int(manifest.get("video_count", 0))

```



```

        skipped = max(0, self._export_requested_count - exported)
        detail = f"; 有 {skipped} 项未导出, 请检查本地文件" if skipped else ""
        self.export_status.setText(f"已导出 {manifest.get('image_count', 0)} 张图片、{manifest.get('video_count', 0)} 个视频{detail} · {session.task_root / 'final'}")
        self.raw_details.setPlainText(json.dumps(manifest, ensure_ascii=False, indent=2))
    else:
        self.export_status.setText("导出失败, 请展开技术详情查看原因")
        self.raw_details.setPlainText(error)
        self.raw_toggle.setChecked(True)
    if self._selected():
        for button in (self.accept_button, self.reject_button, self.pending_button, self.retry_button):
            button.setEnabled(True)
    def _toggle_raw(self, checked: bool) -> None:
        self.raw_details.setVisible(checked)
        self.raw_toggle.setArrowType(Qt.ArrowType.DownArrow if checked else Qt.ArrowType.RightArrow)
    def eventFilter(self, watched, event) -> bool: # noqa: N802 - Qt API
        if watched is self.media_list and event.type() in {QEvent.Type.ShortcutOverride, QEvent.Type.KeyPress}:
            callback = {Qt.Key.Key_A: self.accept_selected, Qt.Key.Key_R: self.reject_selected,
                        Qt.Key.Key_P: self.pending_selected}.get(event.key())
            if callback and not event.modifiers() & (Qt.KeyboardModifier.ControlModifier | Qt.KeyboardModifier.AltModifier | Qt.KeyboardModifier.MetaModifier):
                if event.type() == QEvent.Type.KeyPress:
                    callback()
                    event.accept()
                    return True
            if watched is self.media_list.viewport() and event.type() in {QEvent.Type.Resize, QEvent.Type.Show}:
                self._schedule_thumbnails()
            if watched is self.preview_scroll.viewport() and event.type() == QEvent.Type.Resize:
                QTimer.singleShot(0, self._render_preview)
            return super().eventFilter(watched, event)
    def keyPressEvent(self, event: QKeyEvent) -> None: # noqa: N802 - Qt API
        callback = {Qt.Key.Key_A: self.accept_selected, Qt.Key.Key_R: self.reject_selected,
                    Qt.Key.Key_P: self.pending_selected}.get(event.key())
        if callback:
            callback()
            event.accept()
            return
        super().keyPressEvent(event)
__all__ = ["ReviewPage"]
"""Settings page for non-secret settings, credentials, theme, and FFmpeg."""
from __future__ import annotations
from pathlib import Path
from PySide6.QtCore import Signal
from PySide6.QtWidgets import (
    QComboBox,
    QFileDialog,
    QScrollArea,

```

```

    QFormLayout,
    QGroupBox,
    QHBoxLayout,
    QLabel,
    QLineEdit,
    QPushButton,
    QVBoxLayout,
    QWidget,
)
from .ui import page_header
from ..settings import (
    AppSettings,
    CredentialStore,
    InMemoryCredentialBackend,
    SettingsStore,
    discover_ffmpeg,
)
class SettingsPage(QWidget):
    theme_changed = Signal(str)
    ffmpeg_checked = Signal(object)
    def __init__(
        self,
        settings_store: SettingsStore | None = None,
        credential_store: CredentialStore | None = None,
        parent: QWidget | None = None,
    ):
        super().__init__(parent)
        self.setObjectName("settingsPage")
        self.settings_store = settings_store or SettingsStore()
        if credential_store is None:
            try:
                credential_store = CredentialStore()
            except RuntimeError:
                credential_store = CredentialStore(InMemoryCredentialBackend())
        self.credential_store = credential_store
        self._settings = self._load_settings()
        self.output_edit = QLineEdit(str(self._settings.default_output_path))
        self.theme_combo = QComboBox()
        for caption, value in (("跟随系统", "system"), ("浅色", "light"), ("深色", "dark")):
            self.theme_combo.addItem(caption, value)
        self.theme_combo.setCurrentIndex(max(0, self.theme_combo.findData(self._settings.theme)))
        self.brave_edit = QLineEdit()
        self.brave_edit.setEchoMode(QLineEdit.EchoMode.Password)
        self.x_edit = QLineEdit()
        self.x_edit.setEchoMode(QLineEdit.EchoMode.Password)
        self.twitterapi_edit = QLineEdit()
        self.twitterapi_edit.setEchoMode(QLineEdit.EchoMode.Password)
        self.socialdata_edit = QLineEdit()
        self.socialdata_edit.setEchoMode(QLineEdit.EchoMode.Password)
        self.save_button = QPushButton("保存设置")

```

```

self.save_button.setProperty("primary", True)
self.save_button.clicked.connect(self._save_from_ui)
self.ffmpeg_button = QPushButton("检查视频工具")
self.ffmpeg_button.clicked.connect(self.diagnose_ffmpeg)
self.ffmpeg_status = QLabel("尚未检查")
self.ffmpeg_status.setWordWrap(True)
self.save_status = QLabel()
self.credential_status: dict[str, QLabel] = {}
appearance = QGroupBox("外观")
QFormLayout(appearance).addRow("界面主题", self.theme_combo)
destination = QGroupBox("输出目录")
destination_row = QHBoxLayout(destination)
destination_row.addWidget(self.output_edit, 1)
browse = QPushButton("选择目录")
browse.clicked.connect(self.choose_output)
destination_row.addWidget(browse)
secrets = QGroupBox("API 配置")
secret_form = QFormLayout(secrets)
secret_form.setVerticalSpacing(12)
self._credential_fields = {
    "brave_api_key": self.brave_edit, "x_bearer_token": self.x_edit,
    "twitterapi_api_key": self.twitterapi_edit, "socialdata_api_key": self.socialdata_edit,
}
for name, caption in (("brave_api_key", "Brave 搜索密钥"), ("x_bearer_token", "X 访问凭据"),
    ("twitterapi_api_key", "TwitterAPI.io 密钥"), ("socialdata_api_key", "SocialData 密钥")):
    edit = self._credential_fields[name]
    edit.setPlaceholderText("留空保留已有密钥")
    row = QHBoxLayout()
    row.setSpacing(12)
    row.addWidget(edit, 1)
    state = QLabel()
    state.setObjectName("muted")
    self.credential_status[name] = state
    row.addWidget(state)
    secret_form.addRow(caption, row)
hint = QLabel("密钥仅存入 Windows 凭据库, 已有内容不回填。SocialData 需在新建任务时单独启用。")
hint.setWordWrap(True)
hint.setObjectName("muted")
secret_form.addRow(hint)
self._refresh_credential_status()
ffmpeg = QGroupBox("视频工具")
ffmpeg_layout = QVBoxLayout(ffmpeg)
check_row = QHBoxLayout()
check_row.addWidget(self.ffmpeg_status, 1)
check_row.addWidget(self.ffmpeg_button)
ffmpeg_layout.addLayout(check_row)
self.ffmpeg_details = QGroupBox("技术详情")
self.ffmpeg_details.setCheckable(True)
self.ffmpeg_details.setChecked(False)

```

```

self.ffmpeg_diagnostic = QLabel()
self.ffmpeg_diagnostic.setWordWrap(True)
QVBoxLayout(self.ffmpeg_details).addWidget(self.ffmpeg_diagnostic)
self.ffmpeg_diagnostic.hide()
self.ffmpeg_details.toggled.connect(self.ffmpeg_diagnostic.setVisible)
ffmpeg_layout.addWidget(self.ffmpeg_details)
content = QWidget()
groups = QVBoxLayout(content)
groups.setContentsMargins(0, 0, 6, 0)
for group in (appearance, destination, secrets, ffmpeg):
    groups.addWidget(group)
groups.addStretch()
scroll = QScrollArea()
scroll.setWidgetResizable(True)
scroll.setWidget(content)
layout = QVBoxLayout(self)
layout.setContentsMargins(8, 12, 8, 12)
layout.addWidget(page_header("设置", "选择外观与输出位置, 管理本机 API 凭据。"))
layout.addWidget(scroll, 1)
actions = QHBoxLayout()
actions.addWidget(self.save_status, 1)
actions.addWidget(self.save_button)
layout.addLayout(actions)
def _load_settings(self) -> AppSettings:
    try:
        return self.settings_store.load()
    except ValueError:
        return AppSettings()
@staticmethod
def _theme_value(text: str) -> str:
    return {"system": "system", "light": "light", "dark": "dark"}.get(text, "system")
def _refresh_credential_status(self) -> None:
    for name, label in self.credential_status.items():
        try:
            label.setText("已配置" if self.credential_store.has(name) else "未配置")
        except Exception:
            label.setText("凭据库不可用")
def choose_output(self) -> None:
    selected = QFileDialog.getExistingDirectory(self, "选择默认输出目录", self.output_edit.text())
    if selected:
        self.output_edit.setText(selected)
def _save_from_ui(self) -> None:
    try:
        self.save()
    except Exception:
        # Backend exceptions may contain credentials; never display them.
        self.save_status.setText("保存失败, 请检查输出目录权限或本机凭据库。")
def save(self) -> AppSettings:
    theme = self._theme_value(self.theme_combo.currentData())
    values = self._settings.model_dump()

```

```

        values.update({"default_output_path": Path(self.output_edit.text()), "theme": theme})
        settings = AppSettings.model_validate(values)
        self.settings_store.save(settings)
        self._settings = settings
        for name, edit in self._credential_fields.items():
            if edit.text():
                self.credential_store.set(name, edit.text())
                edit.clear()
        self._refresh_credential_status()
        self.save_status.setText("设置已保存")
        self.theme_changed.emit(theme)
        return settings
def diagnose_ffmpeg(self):
    settings = self._settings
    status = discover_ffmpeg(
        configured_location=settings.ffmpeg_location,
        configured_probe_location=settings.ffprobe_location,
    )
    self.ffmpeg_status.setText("FFmpeg 已就绪" if status.available else "未找到 FFmpeg, 请安装或配置
视频工具。")
    if status.available and not status.ffprobe_path:
        self.ffmpeg_status.setText("FFmpeg 可用, 尚未找到 ffprobe")
    self.ffmpeg_diagnostic.setText(status.diagnostic)
    self.ffmpeg_checked.emit(status)
    return status
__all__ = ["SettingsPage"]
"""Theme helpers for system/light/dark desktop presentation."""
from __future__ import annotations
from PySide6.QtCore import Qt
from PySide6.QtGui import QColor, QPalette, QFont, QFontDatabase
from PySide6.QtWidgets import QApplication
def system_theme(app: QApplication | None = None) -> str:
    """Return the current platform color scheme as ``light`` or ``dark``."""
    app = app or QApplication.instance()
    if app is not None:
        try:
            scheme = app.styleHints().colorScheme()
            if scheme == Qt.ColorScheme.Dark:
                return "dark"
            if scheme == Qt.ColorScheme.Light:
                return "light"
        except AttributeError:
            pass
        try:
            window = app.palette().color(QPalette.ColorRole.Window)
            return "dark" if window.lightness() < 128 else "light"
        except (AttributeError, RuntimeError):
            pass
    return "light"
def _palette(dark: bool) -> QPalette:

```

```

palette = QPalette()
colors = {
    "Window": "#242231" if dark else "#f5f3fb",
    "WindowText": "#f0edf8" if dark else "#302c46",
    "Base": "#302d40" if dark else "#ffffff",
    "AlternateBase": "#383448" if dark else "#efecf9",
    "Text": "#f0edf8" if dark else "#302c46",
    "Button": "#383448" if dark else "#ffffff",
    "ButtonText": "#f0edf8" if dark else "#302c46",
    "Highlight": "#7860b5", "HighlightedText": "#ffffff",
    "Mid": "#504a65" if dark else "#ded9ee",
    "PlaceholderText": "#aaa2bd" if dark else "#827a98",
    "ToolTipBase": "#302d40" if dark else "#ffffff",
    "ToolTipText": "#f0edf8" if dark else "#302c46",
}
for role, color in colors.items():
    palette.setColor(getattr(QPalette.ColorRole, role), QColor(color))
palette.setColor(QPalette.ColorGroup.Disabled, QPalette.ColorRole.Text, QColor("#8c8799"))
palette.setColor(QPalette.ColorGroup.Disabled, QPalette.ColorRole.ButtonText, QColor("#8c8799"))
return palette

STYLE = """
QWidget { color: palette(window-text); }
QMainWindow, QStackedWidget { background: palette(window); }
QLabel { background: transparent; }
QLabel#pageTitle { font-size: 25px; font-weight: 700; }
QLabel#brand { font-size: 19px; font-weight: 700; }
QLabel#muted { color: palette(placeholder-text); }
QFrame#sidebar, QGroupBox, QFrame#card { background: palette(base); border: 1px solid palette(mid); border-radius: 14px; }
QGroupBox { margin-top: 16px; padding: 22px 16px 14px; font-weight: 600; }
QGroupBox::title { subcontrol-origin: margin; left: 18px; padding: 2px 8px; }
QLineEdit, QComboBox, QPlainTextEdit { background: palette(base); border: 1px solid palette(mid); border-radius: 8px; padding: 8px; selection-background-color: palette(highlight); }
QLineEdit:focus, QComboBox:focus { border: 1px solid #9680d5; }
QPushButton { background: palette(button); border: 1px solid palette(mid); border-radius: 8px; padding: 9px 15px; }
QPushButton:hover { border-color: #9680d5; background: palette(alternate-base); }
QPushButton:pressed { background: palette(mid); }
QPushButton[primary="true"] { background: #7860b5; color: white; border: 1px solid #7860b5; font-weight: 600; }
QPushButton[primary="true"]:hover { background: #8b72c5; }
QPushButton:disabled { color: #938caa; background: palette(alternate-base); }
QCheckBox { spacing: 9px; padding: 7px 0; }
QCheckBox::indicator { width: 17px; height: 17px; border: 1px solid palette(mid); border-radius: 4px; background: palette(base); }
QCheckBox::indicator:checked { background: #9680d5; border: 3px solid #cbbbed; }
QListWidget { background: palette(base); border: 1px solid palette(mid); border-radius: 12px; padding: 6px; outline: none; }
QListWidget::item { padding: 10px; border-radius: 8px; }
QListWidget::item:selected { background: palette(alternate-base); color: palette(text); border: 1px solid

```

```

d #b5a4df; }
QListWidget#navigation { background: transparent; border: none; padding: 0; }
QListWidget#navigation::item { padding: 15px 10px; margin: 4px 0; }
QTabWidget::pane { border: none; }
QTabBar::tab { background: palette(base); padding: 10px 15px; border: 1px solid palette(mid); }
QTabBar::tab:selected { color: #9680d5; border-bottom: 3px solid #9680d5; }
QProgressBar { background: palette(base); border: 1px solid palette(mid); border-radius: 7px; text-align
: center; min-height: 23px; }
QProgressBar::chunk { background: #a58cd7; border-radius: 6px; }
QSplitter::handle { background: palette(window); width: 7px; height: 7px; }
QScrollArea { border: none; background: transparent; }
QScrollBar:vertical { background: transparent; width: 10px; margin: 0; }
QScrollBar::handle:vertical { background: palette(mid); border-radius: 5px; min-height: 28px; }
QScrollBar::add-line:vertical, QScrollBar::sub-line:vertical { height: 0; }
QToolTip { padding: 7px; border: 1px solid palette(mid); }
"""

def _prepare_font(app: QApplication) -> None:
    if app.property("toolboxFontReady"):
        return
    from pathlib import Path
    # Qt offscreen environments may not discover installed Windows fonts.
    for filename in ("msyh.ttc", "msyhbd.ttc"):
        path = Path("C:/Windows/Fonts") / filename
        if path.is_file():
            QFontDatabase.addApplicationFont(str(path))
    font = QFont("Microsoft YaHei UI", 10)
    font.setFamilies(["Microsoft YaHei UI", "Microsoft YaHei", "Noto Sans CJK SC", "sans-serif"])
    app.setFont(font)
    app.setProperty("toolboxFontReady", True)

def apply_theme(theme: str = "system", app: QApplication | None = None) -> str:
    """Apply a small palette override and return the effective theme."""
    app = app or QApplication.instance()
    selected = str(theme or "system").casefold()
    if selected not in {"system", "light", "dark"}:
        selected = "system"
    effective = system_theme(app) if selected == "system" else selected
    if app is not None:
        if not app.property("toolboxThemeConnected"):
            app.setProperty("toolboxSystemDark", system_theme(app) == "dark")
            app.styleHints().colorSchemeChanged.connect(
                lambda scheme: _system_changed(app, scheme)
            )
            app.setProperty("toolboxThemeConnected", True)
        if selected == "system":
            effective = "dark" if app.property("toolboxSystemDark") else "light"
        app.setProperty("toolboxTheme", selected)
        if app.style().objectName().casefold() != "fusion":
            app.setStyle("Fusion")
        _prepare_font(app)
        app.setPalette(_palette(effective == "dark"))

```

```

        app.setStyleSheet(STYLE)
    return effective

def _system_changed(app: QApplication, scheme) -> None:
    app.setProperty("toolboxSystemDark", scheme == Qt.ColorScheme.Dark)
    if app.property("toolboxTheme") == "system":
        apply_theme("system", app)
__all__ = ["apply_theme", "system_theme"]
"""Lazy, bounded thumbnail cache used by the review page."""
from __future__ import annotations
from collections import OrderedDict
from collections.abc import Callable
from typing import Generic, TypeVar
T = TypeVar("T")
class ThumbnailCache(Generic[T]):
    """A tiny LRU cache.
    Values are created by the caller supplied factory. This keeps image
    decoding out of the list population path and makes the cache useful for
    both ``QPixmap`` values and light-weight test doubles.
    """
    def __init__(self, max_items: int = 128):
        if int(max_items) < 1:
            raise ValueError("max_items must be positive")
        self.max_items = int(max_items)
        self._values: OrderedDict[str, T] = OrderedDict()
    def get(self, key: str, factory: Callable[[], T] | None = None) -> T | None:
        key = str(key)
        if key in self._values:
            value = self._values.pop(key)
            self._values[key] = value
            return value
        if factory is None:
            return None
        value = factory()
        self.put(key, value)
        return value
    def put(self, key: str, value: T) -> T:
        key = str(key)
        self._values.pop(key, None)
        self._values[key] = value
        while len(self._values) > self.max_items:
            self._values.popitem(last=False)
        return value
    def contains(self, key: str) -> bool:
        return str(key) in self._values
    def clear(self) -> None:
        self._values.clear()
    def __contains__(self, key: object) -> bool:
        return str(key) in self._values
    def __len__(self) -> int:
        return len(self._values)

```



```

@property
def keys(self) -> tuple[str, ...]:
    return tuple(self._values)
__all__ = ["ThumbnailCache"]
"""Small shared widgets and offline vector artwork for the toolbox."""
from __future__ import annotations
from PySide6.QtCore import Qt, QSize
from PySide6.QtGui import QColor, QIcon, QPainter, QPainterPath, QPen, QPixmap
from PySide6.QtWidgets import QLabel, QVBoxLayout, QWidget
def page_header(title: str, subtitle: str) -> QWidget:
    widget = QWidget()
    layout = QVBoxLayout(widget)
    layout.setContentsMargins(0, 0, 0, 8)
    heading = QLabel(title)
    heading.setObjectName("pageTitle")
    text = QLabel(subtitle)
    text.setObjectName("muted")
    text.setWordWrap(True)
    layout.addWidget(heading)
    layout.addWidget(text)
    return widget
def icon(name: str) -> QIcon:
    """Line icons painted locally; no fonts, downloads or data files needed."""
    result = QIcon()
    for scale in (1, 2, 3):
        pixmap = QPixmap(24 * scale, 24 * scale)
        pixmap.setDevicePixelRatio(scale)
        pixmap.fill(Qt.GlobalColor.transparent)
        painter = QPainter(pixmap)
        painter.setRenderHint(QPainter.RenderHint.Antialiasing)
        painter.setPen(QPen(QColor("#9680d5"), 1.7, Qt.PenStyle.SolidLine, Qt.PenCapStyle.RoundCap, Qt.PenJoinStyle.RoundJoin))
        if name in {"new", "file"}:
            painter.drawRoundedRect(5, 3, 14, 18, 2, 2)
            painter.drawLine(9, 11, 15, 11)
            painter.drawLine(12, 8, 12, 14)
        elif name == "progress":
            painter.drawArc(3, 3, 18, 18, 30 * 16, 300 * 16)
            painter.drawLine(12, 6, 12, 12)
            painter.drawLine(12, 12, 16, 14)
        elif name == "review":
            painter.drawRoundedRect(3, 4, 18, 16, 2, 2)
            painter.drawEllipse(6, 7, 3, 3)
            painter.drawLine(4, 18, 12, 11)
            painter.drawLine(12, 11, 20, 18)
        elif name == "history":
            painter.drawRoundedRect(4, 3, 16, 18, 2, 2)
            for y in (8, 12, 16):
                painter.drawLine(8, y, 16, y)
        elif name == "settings":

```

```

        for y, x in ((6, 9), (12, 16), (18, 7)):
            painter.drawLine(3, y, 21, y)
            painter.drawEllipse(x - 2, y - 2, 4, 4)
        else:
            painter.drawLine(4, 12, 20, 12)
            painter.drawLine(12, 4, 12, 20)
        painter.end()
        result.addPixmap(pixmap)
    return result

class Starlight(QWidget):
    """Subtle, entirely local decoration, restricted to the start page."""
    def __init__(self, parent=None):
        super().__init__(parent)
        self.setFixedHeight(38)
        self.setAttribute(Qt.WidgetAttribute.WA_TransparentForMouseEvents)
    def paintEvent(self, event):
        painter = QPainter(self)
        painter.setRenderHint(QPainter.RenderHint.Antialiasing)
        painter.setPen(Qt.PenStyle.NoPen)
        for fraction, y, radius, color in ((.05, 18, 7, "#b5a4df"), (.40, 12, 4, "#aacbea"), (.79, 20, 6
, "#b5a4df"), (.95, 10, 3, "#aacbea")):
            x = self.width() * fraction
            path = QPainterPath()
            path.moveTo(x, y - radius)
            path.lineTo(x + radius / 3, y - radius / 3)
            path.lineTo(x + radius, y)
            path.lineTo(x + radius / 3, y + radius / 3)
            path.lineTo(x, y + radius)
            path.lineTo(x - radius / 3, y + radius / 3)
            path.lineTo(x - radius, y)
            path.lineTo(x - radius / 3, y - radius / 3)
            path.closeSubpath()
            painter.fillPath(path, QColor(color))
__all__ = ["page_header", "icon", "Starlight"]
"""Main window and five-page navigation."""
from __future__ import annotations
from PySide6.QtCore import QTimer, Qt
from PySide6.QtWidgets import (
    QHBoxLayout,
    QVBoxLayout,
    QFrame,
    QLabel,
    QListWidget,
    QListWidgetItem,
    QMainWindow,
    QStackedWidget,
    QWidget,
)
from .controller import DesktopController
from .ui import icon

```

```

class MainWindow(QMainWindow):
    PAGE_NAMES = ("新建任务", "抓取进度", "图片审核", "历史任务", "设置")
    def __init__(self, controller: DesktopController | None = None, parent: QWidget | None = None):
        super().__init__(parent)
        self.setObjectName("mainWindow")
        self.setWindowTitle("周报图片采集与管理软件")
        self.resize(1280, 820)
        self.setMinimumSize(1100, 720)
        self.controller = controller or DesktopController(parent=self)
        self.navigation = QListWidget()
        self.navigation.setObjectName("navigation")
        self.nav_list = self.navigation
        self.navigation.setMinimumWidth(155)
        for name, symbol in zip(self.PAGE_NAMES, ("new", "progress", "review", "history", "settings")):
            self.navigation.addItem(QListWidgetItem(icon(symbol), name))
        self.pages = QStackedWidget()
        self.stacked_widget = self.pages
        self.new_task_page = self.controller.new_task_page
        self.progress_page = self.controller.progress_page
        self.review_page = self.controller.review_page
        self.history_page = self.controller.history_page
        self.settings_page = self.controller.settings_page
        self._close_pending = False
        self._close_signals_connected = False
        for page in (self.new_task_page, self.progress_page, self.review_page, self.history_page, self.settings_page):
            self.pages.addWidget(page)
        self.navigation.currentRowChanged.connect(self.pages.setCurrentIndex)
        self.navigation.setCurrentRow(0)
        sidebar = QFrame()
        sidebar.setObjectName("sidebar")
        sidebar.setFixedWidth(192)
        side = QVBoxLayout(sidebar)
        side.setContentsMargins(16, 26, 16, 18)
        brand = QLabel("周报图片\n采集与管理")
        brand.setObjectName("brand")
        side.addWidget(brand)
        tagline = QLabel("采集 · 筛选 · 审阅")
        tagline.setObjectName("muted")
        side.addWidget(tagline)
        side.addSpacing(25)
        side.addWidget(self.navigation, 1)
        footer = QLabel("让每张配图都有出处")
        footer.setObjectName("muted")
        footer.setWordWrap(True)
        side.addWidget(footer)
        root = QWidget()
        layout = QHBoxLayout(root)
        layout.setContentsMargins(14, 14, 14, 14)
        layout.setSpacing(18)

```

```

        layout.addWidget(sidebar)
        layout.addWidget(self.pages, 1)
        self.setCentralWidget(root)
        self.controller.task_started.connect(lambda _: self.navigation.setCurrentRow(1))
        self.controller.review_loaded.connect(lambda _: self.navigation.setCurrentRow(2))
    def _connect_deferred_close(self) -> None:
        if self._close_signals_connected:
            return
        self._close_signals_connected = True
        for signal in (self.controller.task_finished, self.controller.task_cancelled, self.controller.task_failed, self.controller.history_changed):
            signal.connect(self._schedule_deferred_close)
    def _schedule_deferred_close(self, *_args) -> None:
        if self._close_pending:
            QTimer.singleShot(0, self._finish_deferred_close)
    def _finish_deferred_close(self) -> None:
        if self._close_pending and not self.controller.is_running:
            self._close_pending = False
            self.close()
    def closeEvent(self, event) -> None: # noqa: N802 - Qt API
        if self._close_pending:
            event.ignore()
            return
        if self.controller.is_running:
            self._close_pending = True
            self._connect_deferred_close()
            if self.controller.close():
                self._close_pending = False
                event.accept()
            else:
                event.ignore()
            return
        self.controller.close(timeout_ms=0)
        event.accept()
__all__ = ["MainWindow"]
"""Qt worker boundary for one cooperative pipeline run."""
from __future__ import annotations
from typing import Any
from PySide6.QtCore import QObject, Signal, Slot
from ..pipeline import CancellationRequested, CancellationToken, ProgressEvent
class PipelineWorker(QObject):
    """Run ``PipelineRunner`` on a ``QThread`` without parsing stdout."""
    event = Signal(object)
    progress = Signal(object)
    finished = Signal(object)
    result = Signal(object)
    cancelled = Signal(object)
    failed = Signal(object)
    error = Signal(object)
    def __init__(self, runner: Any, request: Any, token: CancellationToken | None = None):

```

```

    super().__init__()
    self.runner = runner
    self.request = request
    self.token = token or CancellationToken()
def cancel(self) -> None:
    """Request cancellation at the next pipeline checkpoint."""
    self.token.cancel()
def pause(self) -> None:
    """Pause the pipeline at its next cooperative safe point."""
    self.token.pause()
def resume(self) -> None:
    """Resume a pipeline paused at a cooperative safe point."""
    self.token.resume()
def _emit_event(self, value: ProgressEvent) -> None:
    if not isinstance(value, ProgressEvent):
        return
    self.event.emit(value)
    self.progress.emit(value)
@Slot()
def run(self) -> None:
    try:
        value = self.runner.run(
            self.request,
            event_sink=self._emit_event,
            cancellation_token=self.token,
        )
    except CancellationRequested as exc:
        self.cancelled.emit(exc)
    except BaseException as exc: # worker boundary must report all failures
        self.failed.emit(exc)
        self.error.emit(exc)
    else:
        status = str(getattr(value, "status", "")).casefold()
        if self.token.is_cancelled or status == "cancelled":
            self.cancelled.emit(value)
        elif status == "failed":
            self.failed.emit(value)
        else:
            self.finished.emit(value)
            self.result.emit(value)
TaskWorker = PipelineWorker
__all__ = ["PipelineWorker", "TaskWorker"]
"""Review persistence and final-media export contracts."""
from .session import ReviewDecision, ReviewSession, RetryMergeResult
__all__ = ["ReviewDecision", "ReviewSession", "RetryMergeResult"]
"""Defensive review state loading and non-destructive final export."""
from __future__ import annotations
import hashlib
import json
import re

```

```

import shutil
from dataclasses import dataclass
from datetime import datetime, timezone
from enum import Enum
from pathlib import Path
from typing import Any, Iterable
from PIL import Image
from ..domain import (
    ImageCandidate,
    ImageType,
    ScoreBreakdown,
    SourceType,
    VideoCandidate,
    VideoStatus,
    candidate_id_for,
    video_candidate_id_for,
)
from ..delivery.helpers import atomic_json_write, item_names as canonical_item_names, safe_title
from ..delivery.image_conversion import ImageConversionError, ImageConverter
from ..delivery.asset_paths import recover_asset_path
class ReviewDecision(str, Enum):
    ACCEPTED = "accepted"
    PENDING = "pending"
    REJECTED = "rejected"
@dataclass
class RetryMergeResult:
    """Description of one retry merge applied to a review session."""
    news_id: str
    attempt_id: str
    added_image_ids: list[str]
    added_video_ids: list[str]
    @property
    def added_ids(self) -> list[str]:
        return [*self.added_image_ids, *self.added_video_ids]
def _read_json(path: Path, default: Any) -> Any:
    try:
        with path.open("r", encoding="utf-8") as handle:
            value = json.load(handle)
    except (OSError, UnicodeDecodeError, json.JSONDecodeError, TypeError, ValueError):
        return default
    return value
def _as_dict(value: Any) -> dict[str, Any] | None:
    return dict(value) if isinstance(value, dict) else None
def _parse_datetime(value: Any) -> datetime:
    if isinstance(value, datetime):
        parsed = value
    elif isinstance(value, str) and value.strip():
        try:
            parsed = datetime.fromisoformat(value.replace("Z", "+00:00"))
        except ValueError:

```

```

        parsed = datetime.now(timezone.utc)
    else:
        parsed = datetime.now(timezone.utc)
    return parsed if parsed.tzinfo else parsed.replace(tzinfo=timezone.utc)
def _clamp(value: Any, low: float, high: float, default: float = 0.0) -> float:
    try:
        number = float(value)
    except (TypeError, ValueError):
        number = default
    return max(low, min(high, number))
def _stable_fallback(payload: dict[str, Any], prefix: str) -> str:
    encoded = json.dumps(payload, ensure_ascii=False, sort_keys=True, default=str).encode("utf-8")
    return f"{prefix}-{hashlib.sha256(encoded).hexdigest()[:16]}"
def _items(payload: Any, keys: tuple[str, ...]) -> list[dict[str, Any]]:
    if isinstance(payload, list):
        return [item for item in payload if isinstance(item, dict)]
    if not isinstance(payload, dict):
        return []
    for key in keys:
        value = payload.get(key)
        if isinstance(value, list):
            return [item for item in value if isinstance(item, dict)]
    return []
def _review_ids(payload: Any) -> set[str]:
    result: set[str] = set()
    values = payload if isinstance(payload, list) else payload.get("candidates", []) if isinstance(payload, dict) else []
    if not isinstance(values, list):
        return result
    for value in values:
        if isinstance(value, str):
            result.add(value)
        elif isinstance(value, dict) and value.get("id") is not None:
            result.add(str(value["id"]))
    return result
def _safe_title(value: Any, fallback: str) -> str:
    return safe_title(value, fallback)
class ReviewSession:
    """Review images/videos from one output directory.

    All source indexes are read-only. Review decisions are stored in the
    separately managed state path, and final export copies accepted files into
    the task directory.
    """
    _LOW_QUALITY_MIN_WIDTH = 300
    _LOW_QUALITY_MIN_HEIGHT = 300
    _LOW_QUALITY_MIN_PIXELS = 120_000
    _HARD_IMAGE_TYPES = {"logo", "favicon", "icon", "ui", "banner"}
    def __init__(
        self,
        output_dir: Path,

```

```

*,
task_root: Path | None,
state_path: Path,
images: list[ImageCandidate],
videos: list[VideoCandidate],
news_items: list[dict[str, Any]],
source_paths: dict[str, Path],
raw_types: dict[str, str],
review_ids: set[str],
failed_ids: set[str],
ambiguous_image_ids: set[str] | None = None,
):
    self.output_dir = output_dir
    self.task_root = task_root or state_path.parent
    self.state_path = state_path
    self.images = images
    self.videos = videos
    self.news_items = news_items
    self._source_paths = source_paths
    self._raw_types = raw_types
    self._review_ids = review_ids
    self._failed_ids = failed_ids
    self._ambiguous_image_ids = ambiguous_image_ids or set()
    self._decisions: dict[str, ReviewDecision] = {}
    self._image_by_id = {str(item.id): item for item in images}
    self._video_by_id = {str(item.id): item for item in videos}
@classmethod
def from_output(
    cls,
    output_dir: Path | str,
    *,
    state_path: Path | str | None = None,
    task_root: Path | str | None = None,
) -> "ReviewSession":
    root = Path(output_dir).expanduser()
    if root.is_file():
        root = root.parent
    managed_root = Path(task_root).expanduser() if task_root is not None else None
    if state_path is not None:
        state = Path(state_path).expanduser()
    elif managed_root is not None:
        state = managed_root / "review_state.json"
    else:
        # A sibling path is important for imported legacy output: setting a
        # decision must never create a file inside the legacy directory.
        state = root.parent / f".{root.name}.review_state.json"
    image_payload = _read_json(root / "image_index.json", {})
    video_payload = _read_json(root / "video_index.json", {})
    review_payload = _read_json(root / "review_required.json", [])
    failed_payload = _read_json(root / "failed_items.json", [])

```



```

image_items = _items(image_payload, ("candidates", "images", "image_candidates"))
video_items = _items(video_payload, ("videos", "video_candidates", "candidates"))
news_items = cls._news_items(image_payload, video_payload, image_items, video_items)
review_ids = _review_ids(review_payload)
failed_ids = cls._failure_ids(image_payload, video_payload, failed_payload)
images: list[ImageCandidate] = []
videos: list[VideoCandidate] = []
source_paths: dict[str, Path] = {}
raw_types: dict[str, str] = {}
seen_images: dict[str, str] = {}
ambiguous_image_ids: set[str] = set()
seen_videos: set[str] = set()
for raw in image_items:
    candidate, stable_id = cls._coerce_image(raw)
    if candidate is None:
        continue
    previous_news_id = seen_images.get(stable_id)
    if previous_news_id is not None and previous_news_id == candidate.news_id:
        continue
    if previous_news_id is not None:
        # Legacy indexes used URL-only image IDs. Re-key a cross-news
        # collision locally so review choices stay independent.
        ambiguous_image_ids.add(stable_id)
        stable_id = candidate_id_for(candidate.image_url, candidate.news_id)
        object.__setattr__(candidate, "id", stable_id)
        if stable_id in seen_images:
            continue
    seen_images[stable_id] = candidate.news_id
    images.append(candidate)
    raw_types[stable_id] = " ".join(
        [str(raw.get("image_type", "")), *[str(value) for value in raw.get("review_reasons", [])
    ]])

    ).casefold()
    source = cls._resolve_image_path(candidate, root)
    if source is not None:
        source_paths[stable_id] = source
for raw in video_items:
    candidate, stable_id = cls._coerce_video(raw)
    if candidate is None or stable_id in seen_videos:
        continue
    seen_videos.add(stable_id)
    videos.append(candidate)
    raw_types[stable_id] = str(raw.get("media_type", "")).casefold()
    source = cls._resolve_asset_path(raw.get("local_path"), root, candidate.sha256)
    if source is not None:
        object.__setattr__(candidate, "local_path", str(source))
        source_paths[stable_id] = source
session = cls(
    root,
    task_root=managed_root,

```

```

        state_path=state,
        images=images,
        videos=videos,
        news_items=news_items,
        source_paths=source_paths,
        raw_types=raw_types,
        review_ids=review_ids,
        failed_ids=failed_ids,
        ambiguous_image_ids=ambiguous_image_ids,
    )
    session._initialize_decisions()
    return session
@staticmethod
def _news_items(
    image_payload: Any,
    video_payload: Any,
    image_items: list[dict[str, Any]],
    video_items: list[dict[str, Any]],
) -> list[dict[str, Any]]:
    source: list[dict[str, Any]] = []
    for payload in (image_payload, video_payload):
        if isinstance(payload, dict):
            values = payload.get("news_items") or payload.get("items") or []
            if isinstance(values, list):
                source.extend(item for item in values if isinstance(item, dict))
    result: list[dict[str, Any]] = []
    seen: set[str] = set()
    for item in sorted(source, key=lambda value: (int(value.get("sequence", 10**9) or 10**9), str(value.get("news_id", "")))):
        news_id = str(item.get("news_id") or item.get("id") or "")
        if news_id and news_id not in seen:
            result.append(dict(item, news_id=news_id))
            seen.add(news_id)
    for raw in [*image_items, *video_items]:
        news_id = str(raw.get("news_id") or "")
        if news_id and news_id not in seen:
            result.append({"news_id": news_id, "sequence": len(result) + 1, "section": "新作", "title": news_id})
            seen.add(news_id)
    return result
@staticmethod
def _failure_ids(*payloads: Any) -> set[str]:
    result: set[str] = set()
    for payload in payloads:
        values = _items(payload, ("failures",)) if isinstance(payload, dict) else payload if isinstance(payload, list) else []
        for value in values:
            if isinstance(value, dict) and value.get("candidate_id") is not None:
                result.add(str(value["candidate_id"]))
    return result

```

```

@classmethod
def _resolve_asset_path(cls, value: Any, root: Path, expected_hash: str | None) -> Path | None:
    if not value or ".." in Path(str(value)).parts:
        return None
    source = recover_asset_path(value, root, expected_hash)
    if source is not None or any(part.casefold() == ".work" for part in Path(str(value)).parts):
        return source
    # Existing legacy imports can explicitly reference assets outside the
    # index folder. Keep that contract, without using it for stage recovery.
    source = cls._source_path(value, root)
    if source is not None and expected_hash:
        try:
            if hashlib.sha256(source.read_bytes()).hexdigest() != str(expected_hash).casefold():
                return None
        except OSError:
            return None
    return source

@classmethod
def _resolve_image_path(cls, candidate: ImageCandidate, root: Path) -> Path | None:
    resolved: dict[str, Path] = {}
    for name, expected_hash in (
        ("local_path", candidate.output_sha256 or candidate.sha256),
        ("original_path", candidate.original_sha256 or candidate.sha256),
    ):
        recorded = getattr(candidate, name)
        recovered_stage = bool(recorded and any(part.casefold() == ".work" for part in Path(str(recorded)).parts))
        source = cls._resolve_asset_path(recorded, root, expected_hash)
        if source is None:
            object.__setattr__(candidate, name, None)
            continue
        if recovered_stage:
            try:
                with Image.open(source) as image:
                    image.verify()
                # verify() checks the container; load() also forces pixel decode.
                with Image.open(source) as image:
                    image.load()
            except Exception:
                object.__setattr__(candidate, name, None)
                continue
            object.__setattr__(candidate, name, str(source))
            resolved[name] = source
    return resolved.get("local_path") or resolved.get("original_path")

@staticmethod
def _source_path(value: Any, root: Path) -> Path | None:
    if not value:
        return None
    candidate = Path(str(value))
    candidates = [candidate] if candidate.is_absolute() else [root / candidate, candidate]

```

```

    for path in candidates:
        try:
            if path.is_file():
                return path.resolve()
        except OSError:
            continue
    return None
@classmethod
def _coerce_image(cls, raw: dict[str, Any]) -> tuple[ImageCandidate | None, str]:
    data = dict(raw)
    image_url = str(data.get("image_url") or data.get("url") or "")
    news_id = str(data.get("news_id") or data.get("newsId") or "")
    stable_id = str(data.get("id") or (candidate_id_for(image_url, news_id) if image_url else _stable_id_fallback(data, "image")))
    if not news_id:
        return None, stable_id
    score_data = data.get("score") if isinstance(data.get("score"), dict) else {}
    total = _clamp(score_data.get("total", data.get("total", 0.0)), 0.0, 100.0)
    raw_signals = data.get("signals") if isinstance(data.get("signals"), dict) else {}
    score = {
        "relevance": _clamp(score_data.get("relevance", data.get("relevance", total / 100.0)), 0.0,
1.0),
        "freshness": _clamp(score_data.get("freshness", 0.0), 0.0, 1.0),
        "source_trust": _clamp(score_data.get("source_trust", 0.0), 0.0, 1.0),
        "quality": _clamp(score_data.get("quality", data.get("quality", 0.0)), 0.0, 1.0),
        "duplicate_penalty": _clamp(score_data.get("duplicate_penalty", 0.0), 0.0, 1.0),
        "risk_penalty": _clamp(score_data.get("risk_penalty", 0.0), 0.0, 1.0),
        "type_match": _clamp(score_data.get("type_match", 0.0), 0.0, 1.0),
        "total": total,
    }
    payload = {
        "id": stable_id,
        "news_id": news_id,
        "image_url": image_url or f"legacy://{stable_id}",
        "source_url": str(data.get("source_url") or data.get("source") or f"legacy://{stable_id}"),
        "source_type": data.get("source_type", SourceType.UNKNOWN.value),
        "image_type": data.get("image_type", ImageType.UNKNOWN.value),
        "image_type_confidence": _clamp(data.get("image_type_confidence", 0.0), 0.0, 1.0),
        "published_at": _parse_datetime(data.get("published_at")) if data.get("published_at") else None,
        "fetched_at": _parse_datetime(data.get("fetched_at")),
        "width": data.get("width"),
        "height": data.get("height"),
        "mime_type": data.get("mime_type"),
        "byte_size": data.get("byte_size"),
        "sha256": data.get("sha256"),
        "perceptual_hash": data.get("perceptual_hash"),
        "downloadable": bool(data.get("downloadable", False)),
        "local_path": str(data.get("local_path")) if data.get("local_path") else None,
        "original_path": str(data.get("original_path")) if data.get("original_path") else None,
        "original_mime_type": data.get("original_mime_type"),
    }

```

```

        "original_byte_size": data.get("original_byte_size"),
        "original_sha256": data.get("original_sha256"),
        "original_width": data.get("original_width"),
        "original_height": data.get("original_height"),
        "output_mime_type": data.get("output_mime_type"),
        "output_byte_size": data.get("output_byte_size"),
        "output_sha256": data.get("output_sha256"),
        "output_width": data.get("output_width"),
        "output_height": data.get("output_height"),
        "news_source_url": data.get("news_source_url"),
        "parent_source_url": data.get("parent_source_url"),
        "image_alt": data.get("image_alt"),
        "nearby_text": data.get("nearby_text"),
        "selection_reasons": data.get("selection_reasons", []),
        "animated_source": bool(data.get("animated_source", False)),
        "animation_frame_index": data.get("animation_frame_index"),
        "review_reasons": [str(item) for item in data.get("review_reasons", []) if item is not None]
    if isinstance(data.get("review_reasons", []), list) else [],
        "signals": raw_signals,
        "score": score,
        "selected": bool(data.get("selected", False)),
        "curation_status": data.get("curation_status", raw_signals.get("candidate_status", "unselect
ed")),
    }
    try:
        candidate = ImageCandidate.model_validate(payload)
    except (TypeError, ValueError):
        try:
            payload["width"] = int(payload["width"]) if payload["width"] is not None else None
            payload["height"] = int(payload["height"]) if payload["height"] is not None else None
            payload["byte_size"] = int(payload["byte_size"]) if payload["byte_size"] is not None els
e None

            candidate = ImageCandidate.model_validate(payload)
        except (TypeError, ValueError):
            return None, stable_id
    # Domain models derive IDs from URLs. Legacy indexes may have an
    # explicit stable ID, which must remain the review identity.
    object.__setattr__(candidate, "id", stable_id)
    return candidate, stable_id

@classmethod
def _coerce_video(cls, raw: dict[str, Any]) -> tuple[VideoCandidate | None, str]:
    data = dict(raw)
    video_url = str(data.get("video_url") or data.get("url") or "")
    stable_id = str(data.get("id") or (video_candidate_id_for(video_url) if video_url else _stable_f
allback(data, "video")))
    news_id = str(data.get("news_id") or data.get("newsId") or "")
    if not news_id:
        return None, stable_id
    payload = {
        "id": stable_id,

```

```

        "news_id": news_id,
        "source_url": str(data.get("source_url") or f"legacy://{stable_id}"),
        "video_url": video_url or f"legacy://{stable_id}",
        "title": str(data.get("title") or ""),
        "uploader": str(data.get("uploader") or ""),
        "duration_seconds": data.get("duration_seconds"),
        "width": data.get("width"),
        "height": data.get("height"),
        "format": data.get("format"),
        "mime_type": data.get("mime_type"),
        "byte_size": data.get("byte_size"),
        "sha256": data.get("sha256"),
        "local_path": str(data.get("local_path")) if data.get("local_path") else None,
        "downloadable": bool(data.get("downloadable", False)),
        "review_reasons": [str(item) for item in data.get("review_reasons", []) if item is not None]
    if isinstance(data.get("review_reasons", []), list) else [],
        "status": data.get("status", VideoStatus.DISCOVERED.value),
        "failure_reason": data.get("failure_reason"),
        "discovered_via": data.get("discovered_via", "unknown"),
    }
    try:
        candidate = VideoCandidate.model_validate(payload)
    except (TypeError, ValueError):
        return None, stable_id
    object.__setattr__(candidate, "id", stable_id)
    return candidate, stable_id
def _initialize_decisions(self) -> None:
    for candidate in self.images:
        self._decisions[str(candidate.id)] = self._initial_image_decision(candidate)
    for candidate in self.videos:
        self._decisions[str(candidate.id)] = self._initial_video_decision(candidate)
    persisted = self._read_state()
    for media_id, value in persisted.items():
        if media_id in self._decisions and media_id not in self._ambiguous_image_ids:
            try:
                self._decisions[media_id] = ReviewDecision(value)
            except ValueError:
                continue
    # Persist initial state so a fresh task has a durable review contract.
    self._persist()
def _usable(self, candidate: ImageCandidate | VideoCandidate) -> bool:
    source = self._source_paths.get(str(candidate.id))
    return source is not None and source.is_file()
def _hard_filtered(self, candidate: ImageCandidate) -> bool:
    signals = candidate.signals if isinstance(candidate.signals, dict) else {}
    # Semantic type and editorial quality may reject auto-selection, but
    # must not hide an otherwise usable local image from a human reviewer.
    return signals.get("invalid_file") is True
def _initial_image_decision(self, candidate: ImageCandidate) -> ReviewDecision:
    if self._hard_filtered(candidate):

```

```

        return ReviewDecision.REJECTED
    if bool(candidate.selected) and self._usable(candidate):
        return ReviewDecision.ACCEPTED
    if self._usable(candidate):
        return ReviewDecision.PENDING
    return ReviewDecision.REJECTED
def _initial_video_decision(self, candidate: VideoCandidate) -> ReviewDecision:
    if self._usable(candidate) and candidate.status is VideoStatus.DOWNLOADED:
        return ReviewDecision.ACCEPTED
    if self._usable(candidate):
        return ReviewDecision.PENDING
    return ReviewDecision.REJECTED
def _read_state(self) -> dict[str, str]:
    payload = _read_json(self.state_path, {})
    if not isinstance(payload, dict):
        return {}
    values = payload.get("decisions", payload)
    return values if isinstance(values, dict) else {}
def _persist(self) -> None:
    self.state_path.parent.mkdir(parents=True, exist_ok=True)
    atomic_json_write(
        self.state_path,
        {
            "schema_version": 1,
            "updated_at": datetime.now(timezone.utc).isoformat(),
            "decisions": {key: value.value for key, value in sorted(self._decisions.items())},
        },
    )
def decision(self, media_id: str) -> ReviewDecision:
    return self._decisions.get(str(media_id), ReviewDecision.REJECTED)
def set_decision(self, media_id: str, decision: ReviewDecision | str) -> None:
    media_id = str(media_id)
    if media_id not in self._decisions:
        raise KeyError(media_id)
    self._decisions[media_id] = decision if isinstance(decision, ReviewDecision) else ReviewDecision
(decision)
    self._persist()
def _sorted_pending(self, values: Iterable[ImageCandidate]) -> list[ImageCandidate]:
    def key(candidate: ImageCandidate):
        score = candidate.score
        return (
            -(score.total if score else 0.0),
            -(score.source_trust if score else 0.0),
            -((candidate.width or 0) * (candidate.height or 0)),
            str(candidate.id),
        )
    return sorted(values, key=key)
def pending_images(self, news_id: str | None = None) -> list[ImageCandidate]:
    values = [candidate for candidate in self.images if self.decision(str(candidate.id)) is ReviewDe
cision.PENDING]

```

```

        if news_id is not None:
            values = [candidate for candidate in values if candidate.news_id == news_id]
            return self._sorted_pending(values)
    def failed_images(self, news_id: str | None = None) -> list[ImageCandidate]:
        values = [candidate for candidate in self.pending_images(news_id) if str(candidate.id) in self._
failed_ids or bool(candidate.review_reasons)]
        return values
    def accepted_images(self, news_id: str | None = None) -> list[ImageCandidate]:
        values = [candidate for candidate in self.images if self.decision(str(candidate.id)) is ReviewDe
cision.ACCEPTED]
        return [candidate for candidate in values if news_id is None or candidate.news_id == news_id]
    def rejected_images(self, news_id: str | None = None) -> list[ImageCandidate]:
        values = [candidate for candidate in self.images if self.decision(str(candidate.id)) is ReviewDe
cision.REJECTED]
        return [candidate for candidate in values if news_id is None or candidate.news_id == news_id]
    def pending_videos(self, news_id: str | None = None) -> list[VideoCandidate]:
        values = [candidate for candidate in self.videos if self.decision(str(candidate.id)) is ReviewDe
cision.PENDING]
        return [candidate for candidate in values if news_id is None or candidate.news_id == news_id]
    def accepted_videos(self, news_id: str | None = None) -> list[VideoCandidate]:
        values = [candidate for candidate in self.videos if self.decision(str(candidate.id)) is ReviewDe
cision.ACCEPTED]
        return [candidate for candidate in values if news_id is None or candidate.news_id == news_id]
    def _confined_retry_path(self, news_id: str, attempt_id: str) -> Path:
        """Resolve a retry path while rejecting traversal and symlink escapes."""
        if self.task_root is None:
            raise ValueError("retry merge requires a task_root")
        components = (str(news_id), str(attempt_id))
        for component in components:
            value = Path(component)
            if not component or value.is_absolute() or value.name != component or component in {".", "..
"}:
                raise ValueError("retry path must be a single confined component")
        retries_root = (Path(self.task_root) / "retries").resolve()
        lexical = Path(self.task_root) / "retries" / components[0] / components[1]
        resolved = lexical.resolve()
        try:
            resolved.relative_to(retries_root)
        except ValueError as exc:
            raise ValueError("retry path escapes task_root/retries") from exc
        return resolved
    def merge_retry_attempt(self, retry_or_news_id: Any, attempt_id: str | None = None) -> RetryMergeRes
ult:
        """Merge one retry attempt in memory while retaining raw indexes.
        ``retry_or_news_id`` may be an attempt directory, a ``RetryAttempt``
        returned by ``TaskStore``, or a news ID paired with ``attempt_id``.
        Existing stable IDs always win; only new records are appended.
        """
        retry_data = None
        if hasattr(retry_or_news_id, "images") and hasattr(retry_or_news_id, "videos"):

```



```

        retry_data = retry_or_news_id
        news_id = str(retry_data.news_id)
        resolved_attempt_id = str(retry_data.attempt_id)
        root = self._confined_retry_path(news_id, resolved_attempt_id)
        if Path(retry_data.path).resolve() != root:
            raise ValueError("retry path is outside task_root/retries")
        image_items = retry_data.images
        video_items = retry_data.videos
    else:
        if attempt_id is not None:
            news_id = str(retry_or_news_id)
            resolved_attempt_id = str(attempt_id)
            root = self._confined_retry_path(news_id, resolved_attempt_id)
        else:
            raw_path = Path(retry_or_news_id).expanduser()
            if raw_path.is_absolute():
                raise ValueError("retry path must be relative to task_root/retries")
            root = raw_path.resolve()
            retries_root = (Path(self.task_root) / "retries").resolve()
            try:
                root.relative_to(retries_root)
            except ValueError as exc:
                raise ValueError("retry path escapes task_root/retries") from exc
            resolved_attempt_id = root.name
            news_id = root.parent.name
            if not root.is_dir():
                raise FileNotFoundError(root)
            image_items = _items(_read_json(root / "image_index.json", {}), ("candidates", "images", "image_candidates"))
            video_items = _items(_read_json(root / "video_index.json", {}), ("videos", "video_candidates", "candidates"))
        added_images: list[str] = []
        added_videos: list[str] = []
        for raw in image_items:
            candidate, stable_id = self._coerce_image(raw)
            if candidate is None or stable_id in self._image_by_id:
                continue
            self.images.append(candidate)
            self._image_by_id[stable_id] = candidate
            self._raw_types[stable_id] = str(raw.get("image_type", "")).casefold()
            source = self._source_path(raw.get("local_path"), root)
            if source is not None:
                self._source_paths[stable_id] = source
            self._review_ids.add(stable_id)
            self._decisions[stable_id] = self._initial_image_decision(candidate)
            added_images.append(stable_id)
        for raw in video_items:
            candidate, stable_id = self._coerce_video(raw)
            if candidate is None or stable_id in self._video_by_id:
                continue

```

```

        self.videos.append(candidate)
        self._video_by_id[stable_id] = candidate
        source = self._source_path(raw.get("local_path"), root)
        if source is not None:
            self._source_paths[stable_id] = source
            self._decisions[stable_id] = self._initial_video_decision(candidate)
            added_videos.append(stable_id)
        if added_images or added_videos:
            self._persist()
        return RetryMergeResult(news_id, resolved_attempt_id, added_images, added_videos)

def _folder_names(self) -> dict[str, str]:
    # Use the same canonical naming policy as raw output. Manifest order
    # is the editorial appearance order; sorting by global sequence would
    # renumber categories when legacy manifests contain non-monotonic IDs.
    names = canonical_item_names(self.news_items)
    counters = {
        prefix: sum(name.startswith(prefix) for name in names.values())
        for prefix in ("x", "h", "z")
    }
    for candidate in [*self.images, *self.videos]:
        if str(candidate.news_id) not in names:
            # A media record without a corresponding news row belongs to
            # the catch-all weekly namespace, never implicitly to 新作.
            counters["z"] += 1
            names[str(candidate.news_id)] = f"z{counters['z']}"
    return names

def _source_for(self, candidate: ImageCandidate | VideoCandidate) -> Path | None:
    source = self._source_paths.get(str(candidate.id))
    if source is not None and source.is_file():
        return source
    local = getattr(candidate, "local_path", None)
    return self._source_path(local, self.output_dir)

@staticmethod
def _unique_target(target: Path, source: Path) -> Path:
    if not target.exists() or target.resolve() == source.resolve():
        return target
    stem, suffix = target.stem, target.suffix
    index = 2
    while True:
        candidate = target.with_name(f"{stem} ({index}){suffix}")
        if not candidate.exists() or candidate.resolve() == source.resolve():
            return candidate
        index += 1

@staticmethod
def _verified_final_image(candidate: ImageCandidate, source: Path) -> tuple[bytes, str] | None:
    try:
        data = source.read_bytes()
        with Image.open(source) as image:
            actual = (image.format or "").upper()
            image.verify()

```

```

        mime = "image/png" if actual == "PNG" else "image/jpeg" if actual == "JPEG" else None
        extension_matches = source.suffix.casefold() in ({"png"} if actual == "PNG" else {".jpg", ".jpeg"}, {"jpeg"} if actual == "JPEG" else set())
        metadata_matches = candidate.output_mime_type == mime and extension_matches
        hash_matches = not candidate.output_sha256 or hashlib.sha256(data).hexdigest() == candidate.output_sha256

        if mime and metadata_matches and hash_matches:
            return data, mime
        converted = ImageConverter().convert(data, candidate)
        return converted.data, converted.mime_type
    except ImageConversionError:
        return None
    except Exception:
        return None

def export_final(self) -> dict[str, Any]:
    root = self.task_root
    final_root = root / "final"
    image_root = final_root / "images"
    image_root.mkdir(parents=True, exist_ok=True)
    folder_names = self._folder_names()
    files: list[str] = []
    image_ranks: dict[str, int] = {}
    for candidate in self.images:
        if self.decision(str(candidate.id)) is not ReviewDecision.ACCEPTED:
            continue
        source = self._source_for(candidate)
        if source is None or not source.is_file():
            continue
        folder = folder_names.get(str(candidate.news_id), f"x{candidate.news_id}")
        converted = self._verified_final_image(candidate, source)
        if converted is None:
            continue
        data, mime_type = converted
        image_ranks[folder] = image_ranks.get(folder, 0) + 1
        extension = ".jpg" if mime_type == "image/jpeg" else ".png"
        target = image_root / folder / f"{folder}.{image_ranks[folder]:02d}{extension}"
        target.parent.mkdir(parents=True, exist_ok=True)
        target = self._unique_target(target, source)
        target.write_bytes(data)
        files.append(target.relative_to(final_root).as_posix())
    reserved: dict[str, set[str]] = {}
    for candidate in self.videos:
        if self.decision(str(candidate.id)) is not ReviewDecision.ACCEPTED:
            continue
        source = self._source_for(candidate)
        if source is None or not source.is_file():
            continue
        folder = folder_names.get(str(candidate.news_id), f"x{candidate.news_id}")
        target_dir = image_root / folder
        target_dir.mkdir(parents=True, exist_ok=True)

```

```

        extension = source.suffix.casefold() or ".mp4"
        name = _safe_title(candidate.title, str(candidate.id)) + extension
        used = reserved.setdefault(folder, set())
        stem, suffix = Path(name).stem, Path(name).suffix
        index = 2
        while name.casefold() in used or (target_dir / name).exists():
            name = f"{stem} ({index}){suffix}"
            index += 1
        used.add(name.casefold())
        target = target_dir / name
        if target.resolve() != source.resolve():
            shutil.copyfile(source, target)
        files.append(target.relative_to(final_root).as_posix())
    manifest = {
        "schema_version": 1,
        "issue_id": self._issue_id(),
        "files": files,
        "image_count": sum(1 for value in files if not value.casefold().endswith(("mp4", "webm", ".mov", ".mkv", ".avi"))),
        "video_count": sum(1 for value in files if value.casefold().endswith(("mp4", "webm", ".mov", ".mkv", ".avi"))),
    }
    self._atomic_json(final_root / "final_manifest.json", manifest)
    return manifest

def _issue_id(self) -> str:
    for candidate in [self.output_dir / "image_index.json", self.output_dir / "video_index.json"]:
        payload = _read_json(candidate, {})
        if isinstance(payload, dict) and payload.get("issue_id") is not None:
            return str(payload["issue_id"])
    return ""

@staticmethod
def _media_extension(candidate: ImageCandidate | VideoCandidate, source: Path, *, default: str) -> str:
    mime_type = getattr(candidate, "output_mime_type", None) or getattr(candidate, "mime_type", None)

    if isinstance(mime_type, str) and "/" in mime_type:
        suffix = mime_type.split("/", 1)[1].split(";", 1)[0].strip().casefold()
        if suffix == "jpeg":
            suffix = "jpg"
        if suffix and re.fullmatch(r"[a-z0-9]+", suffix):
            return f".{suffix}"
    return source.suffix.casefold() or default

def export(self) -> dict[str, Any]:
    return self.export_final()

@classmethod
def load(cls, output_dir: Path | str, **kwargs: Any) -> "ReviewSession":
    return cls.from_output(output_dir, **kwargs)

@staticmethod
def _atomic_json(path: Path, payload: Any) -> None:
    atomic_json_write(path, payload, ensure_parent=True)

```